



INDIAN INSTITUTE OF TECHNOLOGY
ROORKEE
Department of Computer Science and Engineering

CSN 399: LAB BASED PROJECT

2023 – B.Tech CSE 3rd Year

POST QUANTUM CRYPTOGRAPHY

TEAM:

Deepesh Garg
Jyoti Chauhan
Mayank Kumar

20114032
20114042
20114055

Contents

1	Introduction	2
1.1	Objective	2
1.2	PQC and NIST Competition	2
2	SSL/TLS Protocol: An Overview	4
2.1	TLS Handshake	5
2.2	Different TLS Versions	6
2.2.1	TLS V1.0	6
2.2.2	TLS V1.1	7
2.2.3	TLS V1.2	7
2.2.4	TLS V1.3	8
2.3	TLS V1.3 over TLS V1.2	8
2.4	Classical Cryptography	9
2.5	Post-Quantum Cryptography	10
2.6	KEMTLS	11
2.6.1	What is Key Exchange Mechanism (KEM)	11
2.6.2	KEMTLS vs Post Quantum TLS 1.3	11
3	Setting up the Experiment	13
3.1	Methodology	13
3.2	Certificate Authority and Server Certificates	13
3.2.1	X.509 CERTIFICATE	17
3.3	The Experiment	21
3.3.1	Configuring Openssl with Liboqs	21
3.3.2	Download, configure, build with OQS OpenSSL, and install, the latest Apache web server	22
3.3.3	Building Certificate Authority and Generating Private Key	23
3.3.4	Running the Apache Server	24
4	Comparison and Analysis of TLS connections	28
4.1	Running TLS Demo using Different Algorithms	28
4.1.1	Generating X.509 self-signed certificate using classical authentication al- gorithms(here, rsa)	28
4.1.2	Generating X.509 self-signed certificate using hybrid authentication algo- rithms	34
4.2	Comparing time for TLS handshake with different key exchange algorithms (Per- formance Testing)	37
4.2.1	Comparison Table of number of TLS handshakes using different combi- nations of Signature and Key exchange algorithms	41
5	Conclusion and Future Scope	42

1 Introduction

1.1 Objective

The primary objective of post-quantum cryptography (PQC) is to develop cryptographic techniques and protocols that can resist attacks by quantum computers. Quantum computers have the potential to break many of the current cryptographic algorithms, such as RSA and Elliptic Curve Cryptography (ECC), which are widely used to secure sensitive data. PQC aims to provide a new generation of cryptographic solutions that can withstand quantum computing attacks and ensure the security of sensitive data. The objective is to develop new algorithms and protocols that are secure against both classical and quantum attacks, making them suitable for use in a world where quantum computers are prevalent. Here, we will be working on the technical perspective of Post Quantum Cryptography. Our objective will be to learn the connection between the Certificate Authority and Server Certificates and generate self-signed X.509 certificates. For the practical implementation, we will set up a Quantum-Safe TLS 1.3 Apache web server.

1.2 PQC and NIST Competition

Cryptography plays a critical role in safeguarding the private communication of both individuals and organizations online. By using cryptographic techniques, sensitive information is protected from unauthorized access or manipulation as it travels over the internet or is stored on various online platforms. Whether conducting online purchases or remotely accessing work email, cryptography helps to ensure the security and privacy of our online communications and transactions.

Post-quantum cryptography (PQC) refers to cryptographic techniques and protocols that are designed to be resistant to attacks by quantum computers. The emergence of quantum computing poses a significant threat to many of the current cryptographic algorithms, such as RSA and Elliptic Curve Cryptography (ECC), as they can be broken using Shor's algorithm. The primary goal of PQC is to offer cryptographic solutions that can withstand quantum computing attacks, ensuring the security of sensitive data in a world where quantum computers are widely used. PQC encompasses various cryptographic algorithms and protocols, such as lattice-based cryptography, code-based cryptography, hash-based cryptography, and multivariate cryptography, among others.

Several research efforts are ongoing to develop and standardize PQC algorithms and protocols by organizations such as the National Institute of Standards and Technology (NIST). PQC is a crucial area of research as the power of quantum computers is expected to increase in the future, making it essential to secure sensitive data. Now, for new cryptography to be effective, it must work in conjunction with current protocols like TLS. When designing a new cryptosystem, it is important to consider three key factors:

- The size of encryption keys and signatures - this affects the amount of data transmitted, as smaller keys and signatures require fewer data, while larger ones provide increased security.
- The time required for encryption and decryption, as well as signing and verifying signatures - impacts the efficiency of the communication channel. Faster operations are

preferable, but security should not be compromised.

- The amount of traffic transmitted during encryption, decryption, or signature verification - this can cause network congestion and delays, so new cryptography must be designed to minimize data transmission.

NIST Competition

The NIST competition refers to the competition held by the National Institute of Standards and Technology (NIST) to select new cryptographic algorithms and protocols that are resistant to quantum computing attacks. The competition, which began in 2017, is known as the NIST Post-Quantum Cryptography Standardization Process.

The competition aims to identify and standardize new cryptographic techniques and protocols that can replace the current ones that may become vulnerable to attacks by quantum computers. The competition has attracted many submissions from researchers and organizations around the world, and the selection process is ongoing. The competition is divided into three main phases: the first phase was focused on collecting submissions and identifying the most promising candidates. In the second phase, NIST selected 26 algorithms from the initial submissions to undergo further analysis and evaluation. In the final phase, NIST will select the finalists and standardize them.

The NIST competition is crucial in promoting the development of post-quantum cryptography and ensuring that sensitive data remains secure even in a world where quantum computers are prevalent. Once the competition is complete, the selected cryptographic algorithms and protocols will be available for use by organizations worldwide to secure their data against quantum computing attacks.

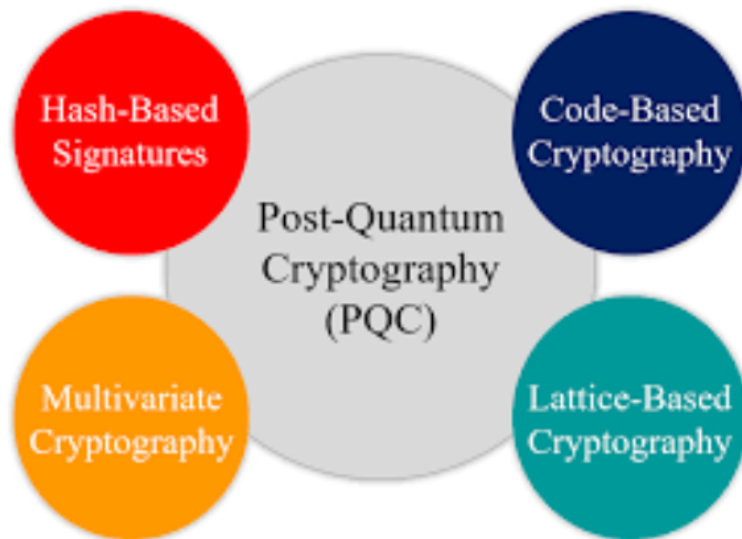
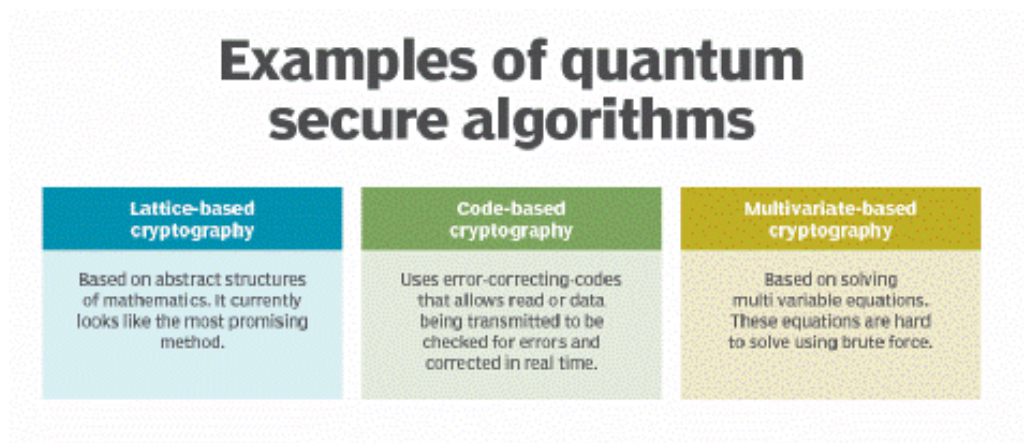


Figure 1: Signature Schemes of Cryptography

After the third round of evaluation, three signature schemes were selected, which are : lattice-based cryptography, code-based cryptography, and multivariate cryptography.

- **Lattice-based cryptography** is built upon mathematical lattices, which are geometric structures that can be used to solve complex mathematical problems. These algorithms provide a high level of security, and their mathematical foundations have been extensively studied, making them a promising candidate for post-quantum cryptography.
- **Code-based cryptography** is based on error-correcting codes, which are used to detect and correct errors in digital data transmission. These algorithms have been in use since the early days of digital communications and are well-established. Code-based cryptography offers high security, but its key sizes are typically large, which can lead to slow operations.
- **Multivariate cryptography** is built on the mathematical problem of solving systems of polynomial equations. These algorithms offer a high level of security and can be implemented with relatively small key sizes, making them efficient. However, they are less studied than lattice-based and code-based cryptography.



2 SSL/TLS Protocol: An Overview

Secure Sockets Layer (SSL) and its successor, Transport Layer Security (TLS), are cryptographic protocols designed to provide secure communication over the internet. These protocols establish a secure channel between two parties, typically a client and a server, ensuring confidentiality and integrity of data transmitted between them. The SSL/TLS protocol uses a combination of symmetric and asymmetric encryption techniques to achieve this. The symmetric encryption algorithm is used to encrypt the data being transmitted, and the asymmetric encryption algorithm is used to establish a secure communication channel. The SSL/TLS protocol also includes several other security features, such as digital certificates, to verify the identity of the communicating parties, and message authentication codes (MACs) to ensure the integrity of transmitted data.

SSL/TLS has evolved over the years, with several versions released since its inception. Currently, TLS 1.2 and 1.3 are the most widely used versions of the protocol. TLS 1.3 is the latest and most secure version, providing improved performance and security compared to its predecessors. TLS 1.3 uses a new key exchange protocol that provides perfect forward

secrecy, which means that even if an attacker manages to obtain the private key of the server, they cannot decrypt previously recorded traffic. In addition, TLS 1.3 supports more secure algorithms and removes support for older, weaker algorithms. Overall, SSL/TLS protocol has become a critical component of secure communication over the internet, and its continued evolution and improvement will be essential for ensuring the security of digital communication in the future.

Typically, TLS is implemented over TCP to encrypt Application Layer protocols like HTTP, FTP, SMTP, and IMAP. However, it can also be used on other transport layer protocols like UDP, DCCP, and SCTP for applications such as VPN and SIP. This implementation is referred to as Datagram Transport Layer Security (DTLS) and is defined in RFCs 6347, 5238, and 6083.

Why do we need TLS?

The absence of Transport Layer Security (TLS) can make sensitive information, such as login credentials, credit card details, and personal data, vulnerable to interception and misuse by unauthorized entities. Moreover, online activities, including browsing patterns, email exchanges, and virtual meetings, can be at risk of surveillance without proper encryption. Implementing TLS in client and server applications safeguards against these threats by encrypting data exchanges with strong algorithms, thus preventing eavesdropping by third parties.

The TLS protocol achieves three main objectives: encryption, authentication, and integrity.

1. Encryption secures the data in transit from unauthorized access.
2. Authentication ensures that the parties communicating are genuine and not imposters.
3. Integrity checks that the data remains unchanged during transmission and has not been tampered with.

2.1 TLS Handshake

TLS handshake is a sequence that initiates a TLS connection between a client device and a web server. The handshake specifies the TLS version and cipher suite that will be used, authenticates the server's identity using its TLS certificate, and generates session keys for encrypting messages between them after the handshake is complete.

- *Encryption* is established using a cipher suite that specifies the shared encryption keys or session keys to be used for the session. Public key cryptography is used to securely generate and exchange a session key. The session key is then used to encrypt and decrypt data transmitted between parties, and is discarded once the session is over.
- *Authentication* is achieved using the server's public key, which is part of its TLS certificate. The certificate is issued by a trusted third party, known as a Certificate Authority, to assert the authenticity of the public key. The server proves its identity to the client using public key encryption, which enables the client to unscramble data encrypted with the server's private key.

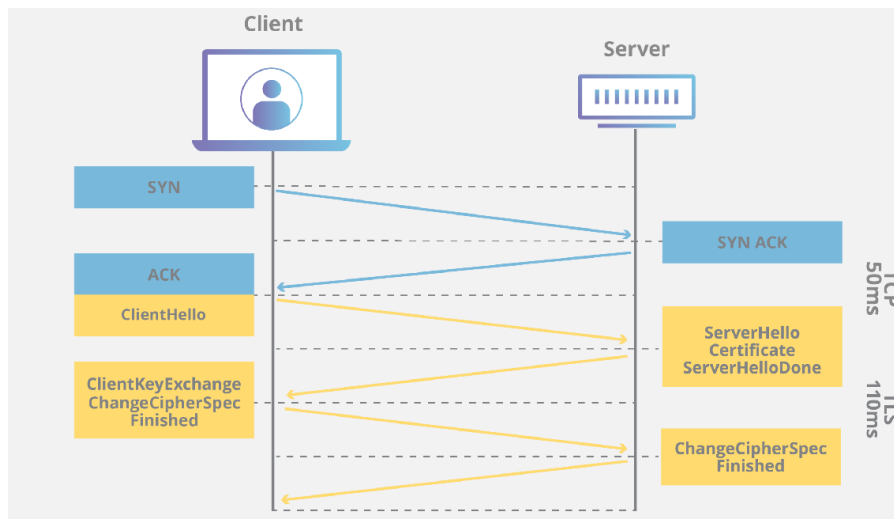
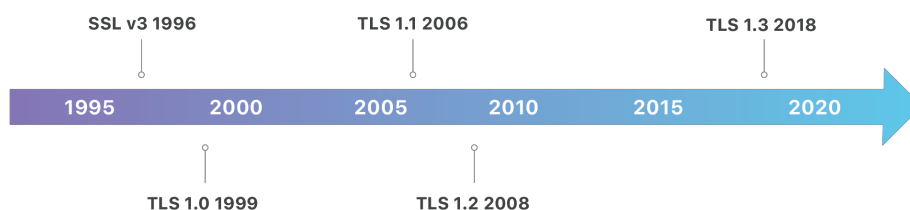


Figure 2: TLS Handshake

- *Integrity* is ensured by signing the data with a message authentication code (MAC). The recipient can verify the MAC to ensure the integrity of the data.

2.2 Different TLS Versions

The IETF is responsible for developing TLS and uses RFCs to codify feedback and ideas. TLS 1.3 is defined by RFC 8446. After a new version of a protocol is released, it's up to browsers and operating systems to integrate support for those protocols. While improved performance and security are incentives for adoption, it can take time for widespread support, as private businesses and consumers may be slow to adopt the latest versions of browsers, applications, and operating systems.



2.2.1 TLS V1.0

TLS 1.0, also known as SSL 3.1 during development, was published in 1999 as an improvement to SSL 3.0. The name change was made to indicate that it was no longer associated with Netscape and to avoid potential trademark issues. Although the differences between SSL 3.0 and TLS 1.0 are not drastic, they are significant enough to prevent interoperability between the two protocols.

TLS 1.0 was designed to provide encryption, authentication, and integrity for online communications, and it quickly became widely adopted. One of the primary features of TLS 1.0

is its support for a wide range of cryptographic algorithms. These algorithms were selected to provide strong encryption and hash functions that can be used to authenticate data.

Despite its early success, TLS 1.0 has several known security vulnerabilities such as POODLE and BEAST attacks. These vulnerabilities were exploited by attackers, leading to several high-profile data breaches. As a result, the IETF released several new versions of TLS to address these vulnerabilities, including TLS 1.1, TLS 1.2, and TLS 1.3.

Today, TLS 1.0 is considered outdated and insecure, and many organizations have stopped using it altogether. Web browsers and operating systems are gradually phasing out support for TLS 1.0 in favor of newer, more secure versions of the protocol.

2.2.2 TLS V1.1

TLS 1.1 was defined in RFC 4346 in April 2006. It is an update from TLS version 1.0. Significant differences in this version include

- added protection against cipher-block chaining (**CBC**) **attacks**,
- and support for **IANA registration** of parameters.

Despite its improvements, support for TLS version 1.1 has been widely deprecated by websites around 2020. This move disabled access to Firefox versions before 24 and Chromium-based browsers before 29. This was due to several security vulnerabilities discovered in both TLS 1.0 and 1.1, which made them less secure than the more recent versions of the protocol. The vulnerabilities were significant enough to prompt the National Institute of Standards and Technology (NIST) to issue guidance recommending that organizations discontinue use of these older versions of TLS.

2.2.3 TLS V1.2

TLS version 1.2 was designed to address the limitations and weaknesses of TLS 1.1. One of the most notable changes in TLS 1.2 is the introduction of the

- **Authenticated Encryption with Associated Data (AEAD) cipher suites**, which provide both confidentiality and integrity protection. The AEAD cipher suites use a single algorithm for both encryption and message authentication, making them more efficient and secure than the earlier cipher suites.
- TLS 1.2 also includes **improvements to the handshake protocol**, which is the process by which the client and server authenticate each other and establish a secure connection. The handshake protocol in TLS 1.2 supports a more secure method for key exchange known as elliptic curve Diffie-Hellman (ECDHE), which provides forward secrecy. This means that even if an attacker were to compromise the private key of the server in the future, they would not be able to decrypt previously recorded communications.

Overall, TLS 1.2 represents a significant improvement over its predecessors in terms of security and efficiency. However, its reliability has been called into question due to the discovery of new vulnerabilities that are similar to those found in the older TLS protocols.

2.2.4 TLS V1.3

TLS 1.3 is the newest iteration of the TLS protocol, which is utilized by HTTPS and other network protocols for encryption. TLS supersedes SSL as the modern standard for encryption. TLS 1.3 improves upon its predecessor, TLS 1.2, by eliminating support for older and less secure cryptographic features and by increasing the speed of TLS handshakes, among other enhancements.

The Internet Engineering Task Force (IETF) published TLS 1.3 in August 2018, a decade after TLS 1.2 was standardized in 2008. TLS 1.3 offers a range of improvements, such as

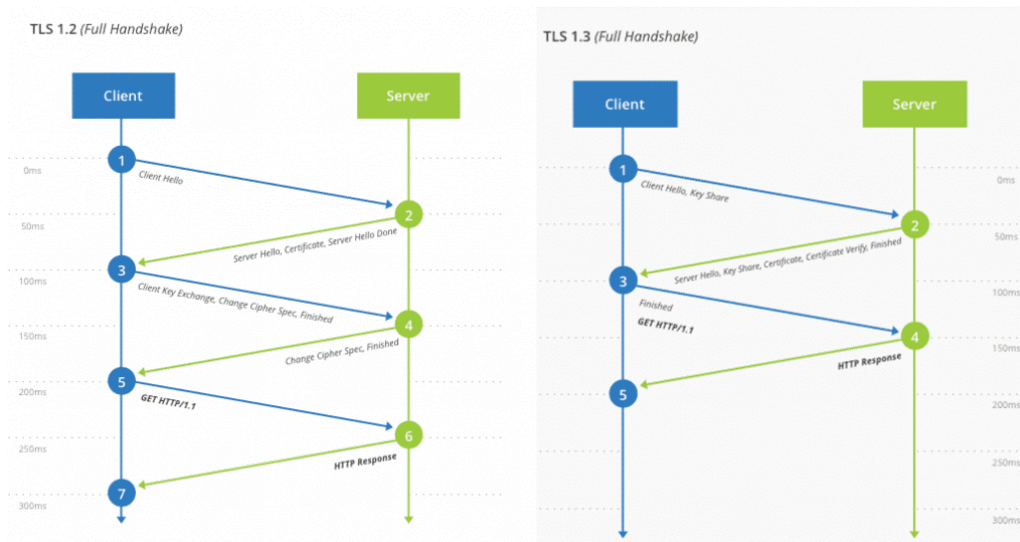
- Zero round-trip time (0-RTT) connection resumption, which allows for faster connections with previously visited servers, and enhanced forward secrecy, which increases the security of past communications even if private keys are compromised.
- Additionally, TLS 1.3 provides a simplified handshake process and improved resistance to attacks, such as the impersonation attacks that target the server authentication process.

Due to its enhanced security and performance, TLS 1.3 has become the standard in many modern browsers and web applications.

2.3 TLS V1.3 over TLS V1.2

TLS 1.3 offers faster and stronger security compared to its predecessor, TLS 1.2. One of the significant changes that contribute to the faster TLS 1.3 is the improvement in the TLS handshake process. TLS 1.3 only requires a single round-trip communication between the client and the server, unlike TLS 1.2 that needs two, effectively shortening the process by a few milliseconds.

- The handshake process in TLS 1.3 starts with the client sending a **clientHello** message to the server.
- This contains a list of supported ciphers based on the client's preference as well as an initial guess about which key algorithm will be used to send a secret key for sharing if required.
- This guesswork eliminates a round-trip, making the process faster.
- Upon receiving the **clientHello** message, the server sends a **serverHello** message.
- The message includes the key, certificate, chosen cipher suite, and the finished message.
- After receiving the server's finished message, the client coordinates with the server on the selected cipher suite to use.



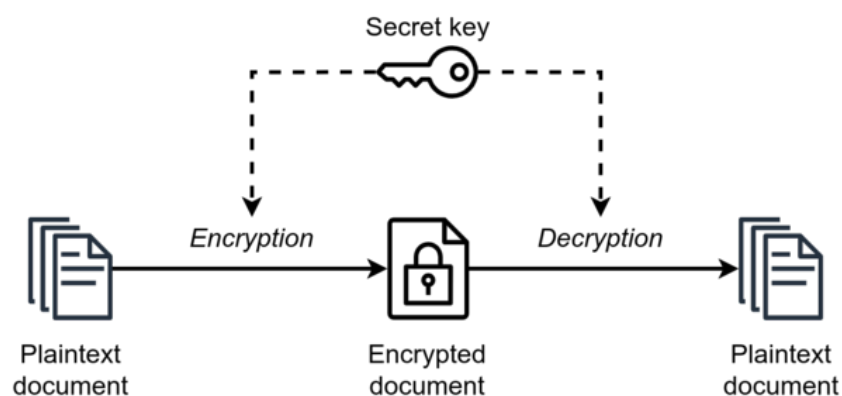
Additionally, if a client has previously connected to a website, the TLS handshake with TLS 1.3 can have zero round trips, resulting in faster HTTPS connections with reduced latency and an improved user experience.

Moreover, TLS 1.3 addresses vulnerabilities discovered in TLS 1.2 and only supports state-of-the-art algorithms with no known vulnerabilities. Additionally, TLS 1.3 offers more privacy by encrypting more parts of the handshake to prevent eavesdropping.

2.4 Classical Cryptography

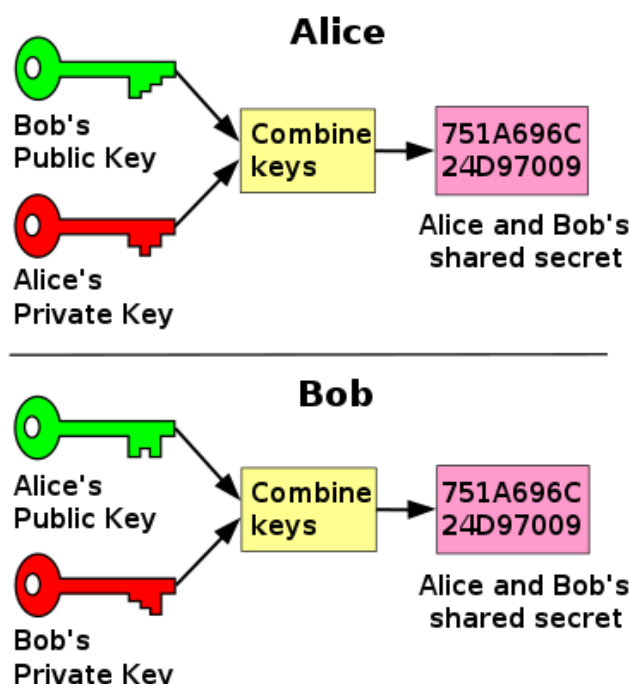
Classical cryptography is the current form of cryptography used worldwide by major companies to achieve security in sending data over internet which is exposed to hackers. It is based on mathematics and primarily relies on the computational difficulty of factorizing large numbers. Hence, the security provided is based on high complexity of the mathematical problem for the instance factorization of the large number. There are two schemes of Classical cryptography:

1. **Symmetric Cryptography-** In Symmetric Cryptography, a single key is used for encrypting and decrypting the data. This key is a private key. Hence this technique requires that the private key is distributed to the senders and the receivers before the communication can begin, which serves as a limitation.



-
2. **Asymmetric Cryptography**- It is also known as public key cryptography as it employs the use of public key. In this scheme of cryptography, a pair of keys (public key and private key) is used to encrypt and decrypt the data.

The sender uses the public key to encrypt the data and the receiver then uses the private key to decrypt the data. The well-known Diffie-Hellman Key Exchange Algorithm uses Public and private keys to generate a shared secret between two communicating parties which can be used as keys for further communication.



2.5 Post-Quantum Cryptography

Shortly, After the development and deployment of classical cryptography schemes, it was discovered that public-key algorithms are vulnerable to attacks by quantum computers. This is due to the fact that these algorithms derive their strength from factoring a large number or the discrete logarithmic problem. As discovered by mathematician Peter Shor, these types of problems can be solved very quickly using a sufficiently strong quantum computer. Hence, the need for algorithms that would stand up to the quantum attacks was felt.

Certain cryptographic hashes such as SHA2, SHA3, BLAKE2, MAC algorithms (like HMAC and CMAK), key-derivation functions (bcrypt, Scrypt, Argon2) are basically quantum-safe. Thus, these can't be broken by sufficiently strong quantum computers.

There is a trade-off between the quantum-level security provided by these algorithms and the size of the public keys generated which increases the response time (which is critical in this modern age of technology.)

This led to the development of KEMTLS which aims at reducing the amount of data required to establish a secure connection and encrypt data over the internet using post-quantum schemes.

2.6 KEMTLS

A variety of experiments have been conducted introducing post-quantum algorithms into the TLS handshake itself. But currently It seems infeasible to replace both the key exchange and signature with post quantum primitives because post-quantum cryptographic primitives are bigger, or slower (or both), than their predecessors.

The following table outlines the difference in public key size for Signature schemes and Key-Exchange Mechanisms for the classical and the post-quantum algorithms. As we can observe, the size is almost 10x (1184 in case of Dilithium- the most widely deployed quantum algorithm - as compared to RSA-2048 – its classical counterpart).

Signature scheme		Public key (bytes)	Signature
RSA-2048	Factoring	272	256
Elliptic curves	Elliptic curve discrete logarithm	32	32
Dilithium	Lattice-based (MLWE/MSIS)	1,184	2,044
Falcon	Lattice-based (NTRU)	897	690
XMSS	Hash-based	32	979
Rainbow	Multi-variate	60,192	66

KEM		Public key (bytes)	Ciphertext (bytes)
RSA-2048	Factoring	272	256
Elliptic curves	Elliptic curve discrete logarithm	32	32
Kyber	Lattice-based (MLWE)	800	768
NTRU	Lattice-based (NTRU)	699	699
Saber	Lattice-based (MLWR)	672	736
SIKE	Isogeny-based	330	330
SIKE compressed	Isogeny-based	197	197
Classic McEliece	Code-based	261,120	128

Also, one key observation to be made is that post-quantum key exchanges have somewhat smaller sizes than post-quantum signatures. We try to replace signatures with key exchanges in some places to save space. This technique is called KEMTLS.

2.6.1 What is Key Exchange Mechanism (KEM)

A KEM is an abstraction of Diffie-Hellman key exchange. In this mechanism, one party creates a public key(pk) and a secret key(sk). This public key is sent to the other side and this side runs the encapsulation mechanism to obtain a cipher text and a shared secret (K). This cipher text is then sent to the former who must use the secret key obtained in the first step in a decapsulation mechanism to obtain the shared secret (K).

2.6.2 KEMTLS vs Post Quantum TLS 1.3

We can estimate the impact of such a replacement on network traffic by simply looking at the sum of the cryptographic objects that are transmitted during the handshake. A typical

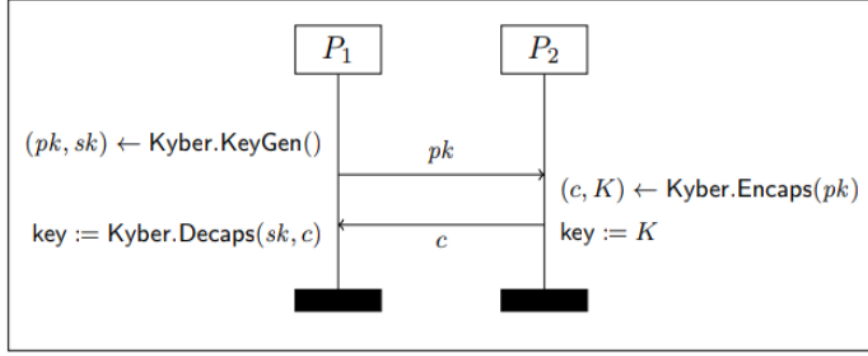


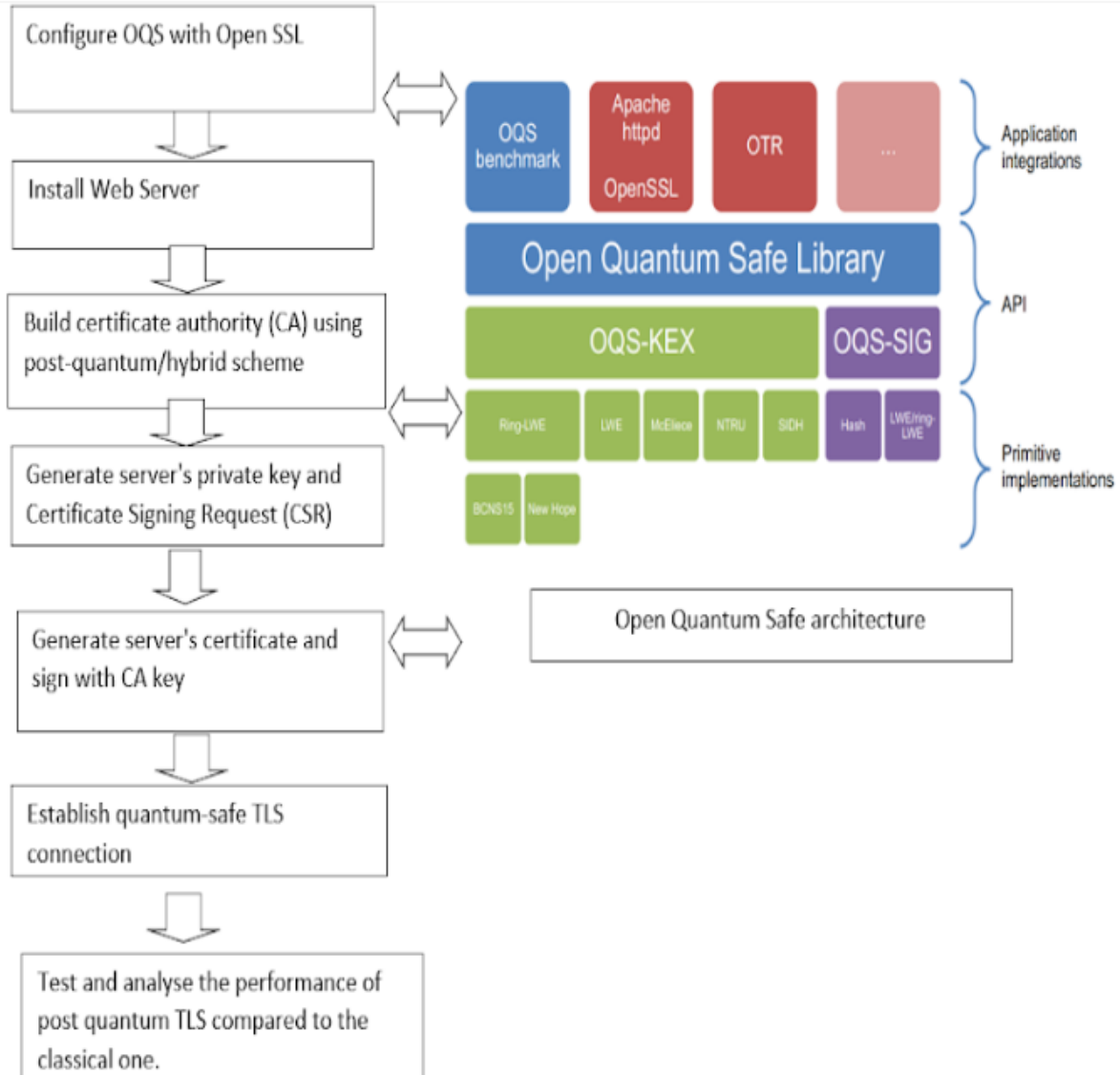
Figure 3: Kyber.KE - Key Exchange Protocol using the Kyber = (KeyGens, Encaps, Decaps) key encapsulation mechanism

TLS 1.3 handshake using elliptic curve X25519 and RSA-2048 would transmit 1,376 bytes, which would correspond to the public keys for key exchange, the certificate, the signature of the handshake, and the certificate chain. If we were to replace X25519 with the post-quantum KEM Kyber512 and RSA by the post-quantum signature Dilithium II, two of the more efficient proposals, the size transmitted data would increase to 10,036 bytes. The increase is mostly due to the size of the post-quantum signature algorithm.

The proposed signature schemes have public keys or signatures much larger than their classical counterparts. For most algorithms, this size increase for post-quantum signatures is bigger than for post-quantum KEMs. The large size of cryptographic material can become an issue in the PQTLS scenario. By replacing the handshake signature with a KEM key exchange, we reduce the size of the data transmitted in the example handshake to 8,344 bytes, using Kyber512 and Dilithium II — a significant reduction.

3 Setting up the Experiment

3.1 Methodology



3.2 Certificate Authority and Server Certificates

A certificate authority (CA) is an organization that stores, signs, and distributes digital certificates. These certificates verify the ownership of a public key by the certificate's named subject. Web browsers use digital certificates to authenticate content transmitted by web servers, establishing trust in online content. The primary objective of a CA is to verify the credibility and reliability of a website, domain, and organization to ensure that users know with whom they are communicating online and whether they can trust them with their data. By issuing a digital certificate for a website, a CA assures users that they are accessing an official

website and not a fraudulent or fake website created by a hacker to steal their information or money.

Key Roles of A Certificate Authority

As an integral part of PKI, a CA plays multiple crucial roles:

- issues digital certificates;
- helps establish trust between communicating entities over the internet;
- verifies domain names and organizations to validate their identities; and
- maintains certificate revocation lists.

How a certificate authority issues a digital certificate

SSL/TLS certificates are responsible for authenticating and securing websites, enabling encrypted connections. To indicate to users that they are visiting a legitimate website, web browsers display a padlock icon. These certificates rely on digital certificates to function, which is where the certificate authority (CA) becomes important as a critical component of PKI. An entity (organization or person) can request a digital certificate from a CA. First, it generates a key pair, which consists of the following:

1. private key, which is always kept a secret and should never be shown to anyone, including the CA; and
2. public key, which is mentioned in the digital certificate the CA issues – the applicant also generates a certificate signing request (CSR), an encoded text file that specifies the information that will be included in the certificate, such as the following:
 - name;
 - additional or alternative domain names, including subdomains;
 - organization; and
 - contact details, e.g., email address.

The CA then confirms the data in the CSR and the applicant's identity after receiving the CSR from the applicant. The CA then creates a digital certificate, uses its private key to digitally sign it, and provides the certificate to the applicant. At this point, the public key of the CA can be used to authenticate this digital certificate, for instance by a web browser. The certificate can also be used by the browser to verify that the digitally signed material was delivered by an authorised party that is in possession of the matching private key and that it has not been changed since that party signed it.

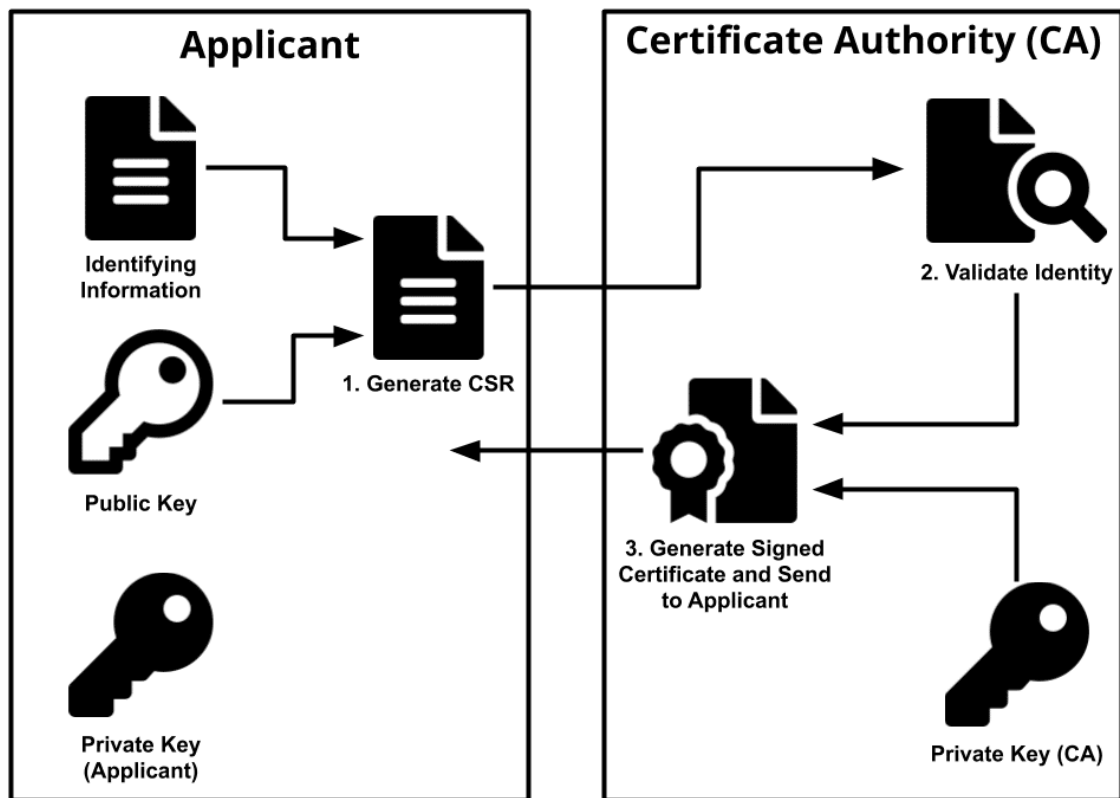


Figure 4: Applicant and Certificate Authority

Chain of Trust

A hierarchy of certificates is used to check the legitimacy of a certificate's issuer in SSL/TLS and other X.509 certificate applications. A chain of trust describes this hierarchy. In a chain of trust, certificates that are higher up the hierarchy are the ones that issue and sign certificates. There are various components to a chain of trust:

1. A trust anchor, represented by the certificate authority (CA) of origin.
2. A minimum of one intermediate certificate that acts as "insulation" between the end-entity certificate and the CA.
3. The end-entity certificate, which confirms the legitimacy of an entity's identity, such as a website, company, or individual.

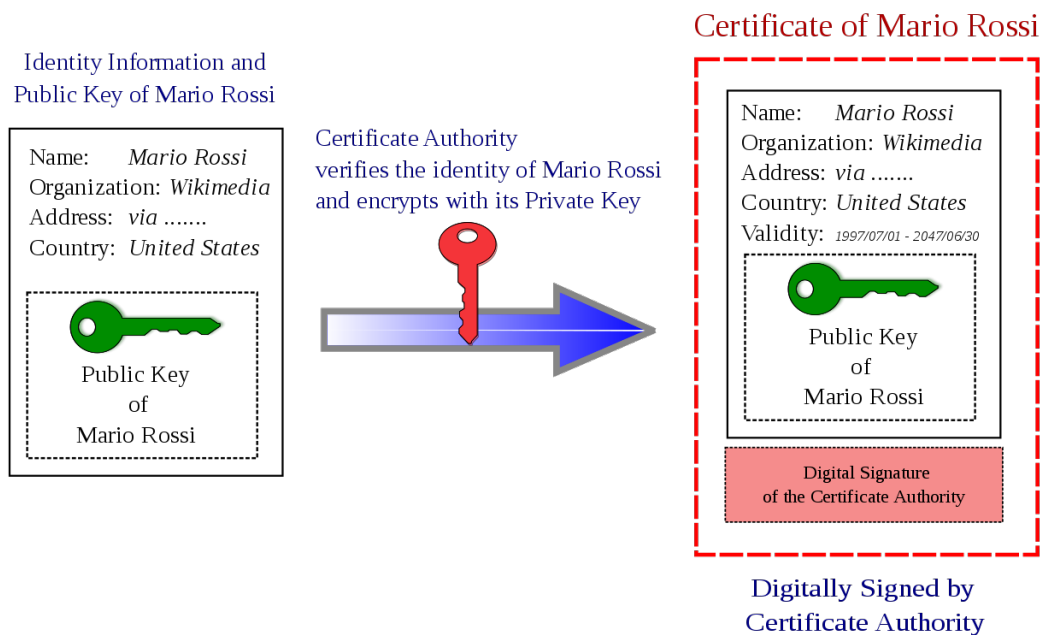


Figure 5: A typical example of issuing a Certificate

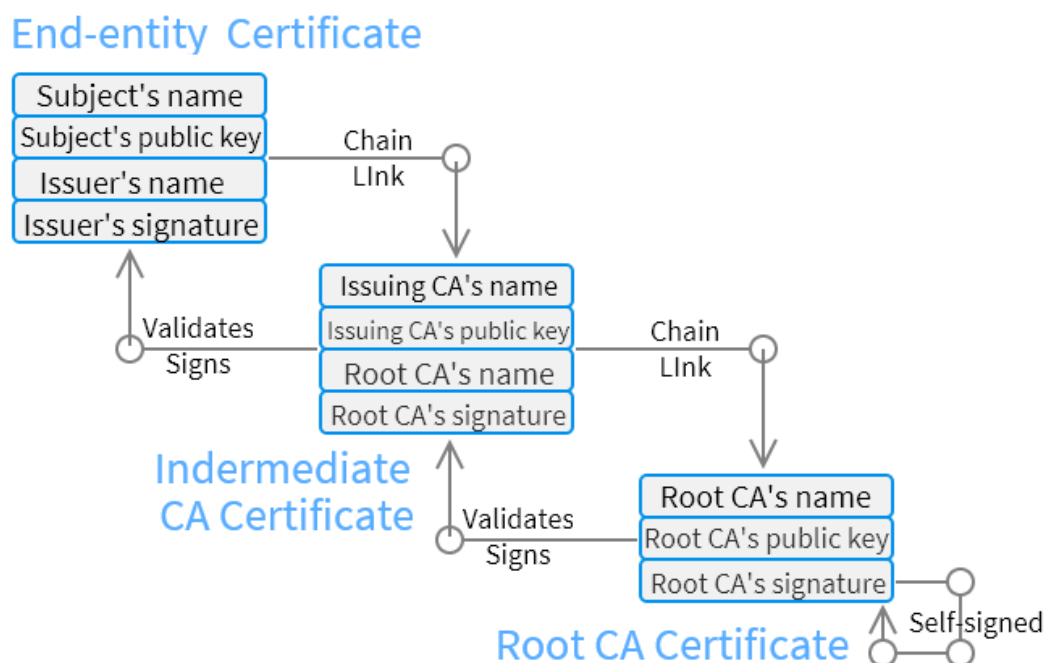


Figure 6: Chain of Trust

3.2.1 X.509 CERTIFICATE

An X.509 certificate is a type of digital certificate that conforms to the widely accepted international X.509 public key infrastructure (PKI) standard. It verifies that a public key is associated with the user, computer, or service identity specified in the certificate.

This type of certificate is used to securely link cryptographic key pairs of public and private keys with websites, individuals, or organizations, and is commonly employed to manage identity and security in computer networking and over the internet. In various internet protocols, X.509 certificates are utilized to prevent malicious websites from deceiving web browsers. Additionally, X.509 certificates are used to secure email, device communications, and digital signatures.

Common applications of X.509 certificates include:

- SSL/TLS and HTTPS for authenticated and encrypted web browsing
- Signed and encrypted email via the S/MIME protocol
- Code signing : enables application developers to add a layer of assurance by digitally signing applications, drivers, and software programs so that end users can verify that a third party has not altered or compromised the code they receive.
- Document signing
- Client authentication
- Government-issued electronic ID

Certificate Fields and Extensions

X.509 certificate fields contain information about the identity that the certificate is issued to as well as the identity of the issuer CA. The standard fields include:

1. **Version** – the X.509 version that applies to the certificate
2. **Serial number** – the unique serial number identifier provided by the CA that distinguishes the certificate from others
3. **Algorithm information** – the cryptographic algorithm used by the issuer to sign the certificate
4. **Issuer distinguished name** – the name of the CA issuing the certificate
5. **Validity period of the certificate** – the start/end date and time it's valid and can be trusted
6. **Subject distinguished name** – the name of the identity the certificate is issued to
7. **Subject public key information** – the public key associated with the identity

The X.509 version 3 defined a number of extensions in addition to its basic information fields to facilitate extended ways client programmes might use the internet. The Subject Alternative Name and Key Usage X.509 certificate extensions are the two most popular ones in use right now.

- Other identities may be linked to a certificate's public key using the **Subject Alternative Name extension**. Other domains, DNS names, email addresses, and IP addresses may be included. Multiple-domain certificates provided by CAs are usually referred to as SAN certificates based on this extension.
- **Key Usage** limits the use of the keys to particular purposes such as "signing-only."



www.ssl.com
Issued by: SSL.com EV SSL Intermediate CA RSA R3
Expires: Saturday, April 17, 2021 at 5:15:06 PM Central Daylight Time
✔ This certificate is valid

▼ Details

Subject Name	
Country or Region	US
State/Province	Texas
Locality	Houston
Organization	SSL Corp
Serial Number	NV20081614243
Common Name	www.ssl.com
Postal Code	77098
Business Category	Private Organization
Street Address	3100 Richmond Ave
Inc. State/Province	Nevada
Inc. Country/Region	US

Issuer Name	
Country or Region	US
State/Province	Texas
Locality	Houston
Organization	SSL Corp
Common Name	SSL.com EV SSL Intermediate CA RSA R3

Serial Number	72 14 11 D3 D7 E0 FD 02 AA B0 4E 90 09 D4 DB 31
Version	3
Signature Algorithm	SHA-256 with RSA Encryption (1.2.840.113549.1.1.11)
Parameters	None
Not Valid Before	Thursday, April 18, 2019 at 5:15:06 PM Central Daylight Time
Not Valid After	Saturday, April 17, 2021 at 5:15:06 PM Central Daylight Time

Figure 7: www.ssl.com's SSL/TLS certificate, as shown in Google Chrome

Revocation

X.509 certificates that must be invalidated before their Not Valid After date may be revoked. As mentioned above, RFC 5280 profiles certificate revocation lists (CRLs), time-stamped lists of revoked certificates that can be queried by browsers and other client software.

How Do I Get an X.509 Certificate?

To deploy X.509 certificates, it is essential to have a trusted certification authority (CA) or agent responsible for issuing certificates and publishing public keys associated with private keys held by individuals. Without a trustworthy CA, it would be impossible to verify that the correct public key, and not one belonging to a malicious actor intending to intercept sensitive information, is being used by the sender. This verification is critical in preventing sensitive information from falling into the wrong hands and being used for harmful purposes.

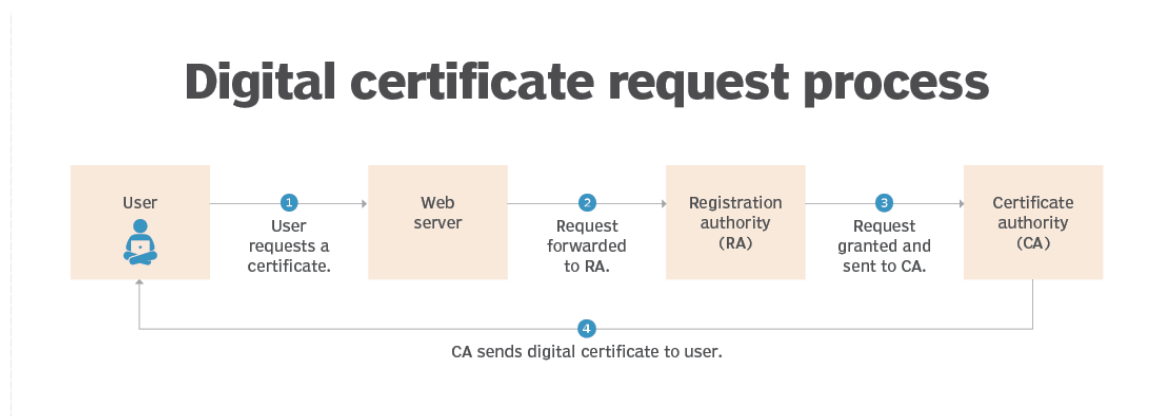


Figure 8: Digital Certificate Request Process

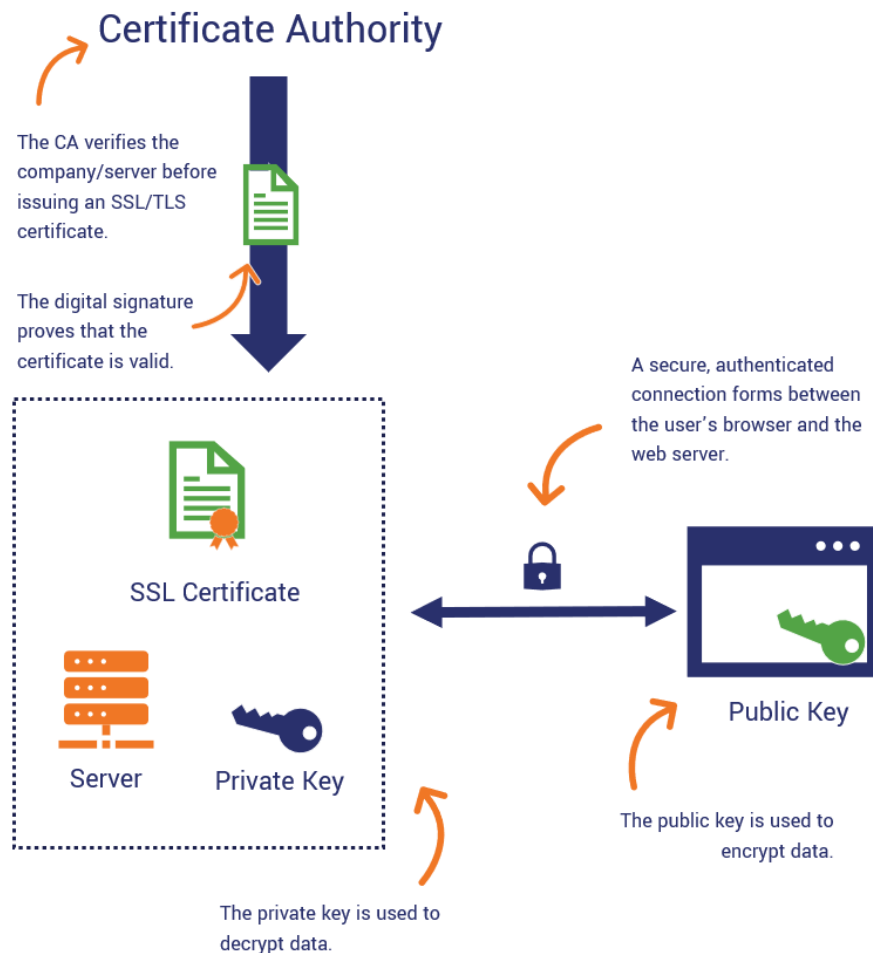
Validation of certificates in browsers

When a browser connects to a secure website using HTTPS (HTTP over SSL/TLS), it verifies the identity of the website by checking the SSL/TLS certificate presented by the server. This certificate is issued by a Certificate Authority (CA), which is a trusted third-party organi-

zation that verifies the identity of the website owner and issues a digital certificate containing the website's public key.

To ensure the authenticity of the SSL/TLS certificate, the browser checks if it has been issued by a trusted CA. Browsers come with a pre-installed list of trusted CAs, also known as the root CA certificates. These certificates are issued by well-known and reputable organizations such as Comodo, DigiCert, and Let's Encrypt, and are used to verify the SSL/TLS certificates presented by websites. When a browser connects to a secure website, it checks the SSL/TLS certificate to ensure that it is valid, not expired, and issued by a trusted CA. If the certificate passes these checks, the browser establishes a secure connection with the server, and any data sent between the browser and the server is encrypted and protected from eavesdropping and tampering.

In summary, browsers use CA certificates to verify the identity of the SSL/TLS certificates presented by websites and establish a secure connection with them. This process helps to ensure the confidentiality, integrity, and authenticity of the data exchanged between the browser and the server.



3.3 The Experiment

In the experiment, we will be building a prototype client-server build which will protect communications against both classical and quantum threats. The various features it supports are:

1. TLS Version 1.3
2. For signing its CA and server certificates, it uses a hybrid signature scheme with Level 5 security, combining NIST's P-521 elliptic curve with Falcon 1024, a Round 3 finalist in NIST's post-quantum standardization process.
3. For its TLS handshake, it uses a hybrid key-exchange method with Level 5 security, combining NIST's P-521 elliptic curve with Firesaber, also a Round 3 finalist algorithm.

Additionally, It can be configured to select other Level 5 secure NIST Round-3 finalist hybrid signature and key exchange schemes, should the user desire to do so.

The steps to the experiments are described below:

3.3.1 Configuring Openssl with Liboqs

1. Cloning Openssl into the local machine

- sudo apt install cmake gcc libtool libssl-dev make ninja-build git -y
- git clone --branch OQS-OpenSSL_1_1_1-stable https://github.com/open-quantum-safe/openssl.git

```
deepesh@deepesh:~$ git clone --branch OQS-OpenSSL_1_1_1-stable https://github.com/open-quantum-safe/openssl.git
Cloning into 'openssl'...
remote: Enumerating objects: 390746, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (31/31), done.
Receiving objects: 18% (70818/390746), 33.26 MiB | 3.33 MiB/s
```

2. Integrating Liboqs with Openssl

- git clone --branch main https://github.com/open-quantum-safe/liboqs.git
- cd liboqs
- mkdir build cd build
- cmake -GNinja -DCMAKE_INSTALL_PREFIX= /openssl/oqs ..
- ninja
- ninja install

```
deepesh@deepesh:~$ git clone --branch main https://github.com/open-quantum-safe/liboqs.git
Cloning into 'liboqs'...
remote: Enumerating objects: 29284, done.
remote: Counting objects: 100% (481/481), done.
remote: Compressing objects: 100% (266/266), done.
```

3. Configuring Quantum safe algorithms

```
-cd /openssl
-./Configure no-shared linux-x86_64
-DOQS_DEFAULT_GROUPS=p521_kyber1024:p521_kyber90s1024:p521_ntru_hps40961229:p521_ntru_
-make -j 1
-sudo make install
```

```
deepesh@deepesh:~/openssl$ ./Configure no-shared linux-x86_64 -DOQS_DEFAULT_GROUPS=\"p521_kyber1024:p521_kyber90s1024\" -l
m
Configuring OpenSSL version 1.1.1t (0x1010114fL) for linux-x86_64
Using os-specific seed configuration
Creating configdata.pm
Creating Makefile

***
***      OpenSSL has been successfully configured      ***
***
***  If you encounter a problem while building, please open an ***
***  issue on GitHub <https://github.com/openssl/openssl/issues> ***
***  and include the output from the following command: ***
***
***      perl configdata.pm --dump ***
***
***  (If you are new to OpenSSL, you might want to consult the ***
***  'Troubleshooting' section in the INSTALL file first) ***
***
*****
```

3.3.2 Download, configure, build with OQS OpenSSL, and install, the latest Apache web server

```
-wget https://dlcdn.apache.org/httpd/httpd-2.4.57.tar.gz
-tar -xvf httpd-2.4.57.tar.gz
-wget https://dlcdn.apache.org/apr/apr-1.7.3.tar.gz
-tar -xvf apr-1.7.3.tar.gz
-wget https://dlcdn.apache.org/apr/apr-util-1.6.3.tar.gz
-tar -xvf apr-util-1.6.3.tar.gz
-mv apr-1.7.3 httpd-2.4.57/srclib/apr
-mv apr-util-1.6.3 httpd-2.4.57/srclib/apr-util
-sudo apt install libpcre3-dev libpcre3
-wget https://github.com/libexpat/libexpat/releases/download/R_2_4_1/expat-2.4.1.tar.gz
-tar -xvf expat-2.4.1.tar.gz
-cd expat-2.4.1
-./configure
-make
-sudo make install
-cd /httpd-2.4.57
-./configure --with-ssl= /openssl/apps/openssl --with-expat=/usr/local/include
-make
-sudo make install
```

```

deepesh@deepesh:~$ wget https://dlcdn.apache.org/httpd/httpd-2.4.57.tar.gz
--2023-04-11 17:47:15-- https://dlcdn.apache.org/httpd/httpd-2.4.57.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9773385 (9.3M) [application/x-gzip]
Saving to: 'httpd-2.4.57.tar.gz'

httpd-2.4.57.tar.gz          100%[=====>]    9.32M  5.83MB/s

```

```

deepesh@deepesh:~$ wget https://dlcdn.apache.org/apr/apr-1.7.3.tar.gz
--2023-04-11 17:50:57-- https://dlcdn.apache.org/apr/apr-1.7.3.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1097690 (1.0M) [application/x-gzip]
Saving to: 'apr-1.7.3.tar.gz'

apr-1.7.3.tar.gz          100%[=====>]    1.05M  2.67MB/s   in 0.4s

2023-04-11 17:50:58 (2.67 MB/s) - 'apr-1.7.3.tar.gz' saved [1097690/1097690]

```

3.3.3 Building Certificate Authority and Generating Private Key

1. Copy CA certificate to system trust store and add it as a trust anchor for the web browser

```

deepesh@deepesh:~/httpd-2.4.57$ cd ~
deepesh@deepesh:~$ openssl/apps/openssl req -x509 -new -newkey p521_falcon1024 -keyout p521_falcon1024_CA.key -out p521_falcon1024_CA.crt -nodes -subj "/CN=oqstest CA" -days 365 -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_CA.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl req -new -newkey p521_falcon1024 -keyout p521_falcon1024_srv.key -out p521_falcon1024_srv.csr -nodes -subj "/CN=localhost" -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_srv.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl x509 -req -in p521_falcon1024_srv.csr -out p521_falcon1024_srv.crt -CA p521_falcon1024_CA.crt -CAkey p521_falcon1024_CA.key -CAcreateserial -days 365
Signature ok
subject=CN = localhost
Getting CA Private Key

```

2. Generating server's hybrid private key and Certificate Signing Request (CSR)

```

deepesh@deepesh:~/httpd-2.4.57$ cd ~
deepesh@deepesh:~$ openssl/apps/openssl req -x509 -new -newkey p521_falcon1024 -keyout p521_falcon1024_CA.key -out p521_falcon1024_CA.crt -nodes -subj "/CN=oqstest CA" -days 365 -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_CA.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl req -new -newkey p521_falcon1024 -keyout p521_falcon1024_srv.key -out p521_falcon1024_srv.csr -nodes -subj "/CN=localhost" -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_srv.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl x509 -req -in p521_falcon1024_srv.csr -out p521_falcon1024_srv.crt -CA p521_falcon1024_CA.crt -CAkey p521_falcon1024_CA.key -CAcreateserial -days 365
Signature ok
subject=CN = localhost
Getting CA Private Key

```

3. Generate server's hybrid certificate and sign with CA hybrid key

```

deepesh@deepesh:~/httpd-2.4.57$ cd ~
deepesh@deepesh:~$ openssl/apps/openssl req -x509 -new -newkey p521_falcon1024 -keyout p521_falcon1024_CA.key -out p521_falcon1024_CA.crt -nodes -subj "/CN=oqstest CA" -days 365 -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_CA.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl req -new -newkey p521_falcon1024 -keyout p521_falcon1024_srv.key -out p521_falcon1024_srv.crt -nodes -subj "/CN=localhost" -config ~/openssl/apps/openssl.cnf
Generating a p521_falcon1024 private key
writing new private key to 'p521_falcon1024_srv.key'
-----
deepesh@deepesh:~$ openssl/apps/openssl x509 -req -in p521_falcon1024_srv.csr -out p521_falcon1024_srv.crt -CA p521_falcon1024_CA.crt -CAkey p521_falcon1024_CA.key -CAcreateserial -days 365
Signature ok
subject=CN = localhost
Getting CA Private Key

```

4. Performing a basic test of TLS connectivity

```

deepesh@deepesh:~$ openssl/apps/openssl s_server -cert p521_falcon1024_srv.crt -key p521_falcon1024_srv.key -www -tls1_3
Using default temp DH parameters
ACCEPT

```

```

deepesh@deepesh:~$ openssl/apps/openssl s_client -groups p521_kyber1024 -CAfile p521_falcon1024_CA.crt -connect localhost
CONNECTED(00000003)
Can't use SSL_get_servername
depth=1 CN = oqstest CA
verify return:1
depth=0 CN = localhost
verify return:1
---
Certificate chain
 0 s:CN = localhost
  i:CN = oqstest CA
  ---
Server certificate
-----BEGIN CERTIFICATE-----
MIINmzCCAgCFfnJA6VVtBvQ0neNjLGSFyS5GCC5MACGBSvODwMFMbUxEzARBgNV
BAMC9xc3Rlc3Q0Q0EwHhcNMjWnDEyMDkwmjEwHhcNMjQwNDEyMDkwmjEwWjAU
MRlWAEYDVQ0DDALsb2NhbGhvc3QwggEYMAcGBSvODwMFA4IHwAAACFBAELUQ0o
EpAwRfuL9kGRHdLfemuK5IvWwJvECy0LiOARb5MnmY04fqdHt46CLr+9wDlj6JEF
p0ZZ+ckFtypxqmIQtgBF+3KdMHqjsgR2h+lt/EEuG9Knk4cQ96yVkyTDEu00WQ/
bZiFHdKKrQKVfDeCP+HZbMRaHPN6TGw/mcJ0o6TeeApmKnsRdWNfEp2GZIEySto
gJLABxludBd5mhseMESCT2oNMy4aorCdwM2ssQRFY1wcXD0m2xbuWih51MbnJbyn
KvDjySVqc2Z8R0MQCEKenj/lveFVjgak9Jvv0JwE378mWcE5CMGCNoZtmx4yKeU
nJGKcLySGDnXbtNgFiqKVvGbqi/XjZpv0GbmTaIaPwDrheF+EHFneMvvY9Ted+EQ
8VaQaunwKaRQBLngvoXoXWlbcAGNKj382pckRRN4kbzhIRtgeHStQdnt0x+IX6Xh
G0FPJo4yWgBeHk5fu4tVzEQcBNLeGC2B1KOKL6vKvY35YE3qi5ImuxNVR3YHjo5C
/p/HfACekfSNPSYwRoLuxeZPT07UCZeGMDnCLAZfmrIeQe+icZELugR5ZYrQ0GJ
l4FH0G8SozOLCA5jdAjymKJnrcA9N5GbNjZGHnSMIzRj1HgEL10MgQJyQK4Y+DPM
YCCoIToUgINTYtLkIzzyd+9RgKaMyFKBR2wvCfrmqRvGtKwVkrAvrXDlnkHJXTse
icaR+4J/mrR2EvfXwC68oqRUVLMmSVqLAQpkXLB0SMq08Fgi3KuYUF0aqSUUYcTJ
v0LH9UzLJonuzKWe57SgfpvZ0LEgrppMualh2iWswM9yLSESLh4AZ+DdHw90FSp
koU4sQeW2JKqgNRakMnxcuFpa0XeIn2C6tYF4o6ioSTLV8qAXqMDMBviIDTjJtAR
Cw5BRAPU3sQMLt6n11JLJIDIL0juLgaZPjpwNk4ssTD8VuIzZoVzQ9jG6cOXqL
t0NNe2BT6+FTAhDweBP1LZ1DbesmuhoXcrr7uxySD1qy3WnU2r9dclIG98yJHx7a
NJjlv0c0U1Kn70BnxjYSYsgSKHWDJ1UuP60jyF8Qa9099Ari1pJFEZlzoaXyALKw2
QaUmVld/wYDMMyN2kKpPchYQpXtOX+DrhXSRcZUZORomSgaC5YytKwg4K9l2muJv
bjaw2dI5IC2PUUdbgfFu2/dQD7Ca+Rle1PxneUGUT8fMnXl98WxYd3sddhMqIW9dd
ng2FUCPsCEl2mqeBrs6eNcjXUwxJWR5IOr4DERT2WWXBRJNRR4Yjv44dq4zUPXc
4W88prbMvLnTROX3cYBDXZ4UGrdF2WWJzLXzn70+NADU1SKLY1RMtphKtAPLSD
nfSftgVrSpnyMVpmQWgQizRjtnlqUgymARgF6rhSjH61JZCGkIkz7WhxjNzZ3JI
Gy56hgIWF/xpJ77e0Nl4g9U2wDb280gbv4B/IFWiXDNd3QbhwUn8WDLSeWooFQ
EuZVapLKLTLOMwa0PsiY1PLbtGtyD2cBCIRrW9FR7zZZrIkSak4x8o3G9lbgVhaoG
XTQ/jctSBBcatlYgvRElQUOvGq49sBkwqkvGF0VJQqDLKJ5x0xEG56R92d5EJdf
OivH7IubP01AKyPMkI04TLGrmGJXoLR3B1ErEWZqILt0LYQMPeE/YxUZ/VirxYrE
MzDVWsmRhLANLEHYL9KhYu8x5Yyf4meZMNTG0edR0GrpEl1nCFZkKwo0YJhWzhs
ML/SgPebwqIZSqHrhYnUAMaZ8RCruEZY5atYVJBuBP8UgYFVu/x5w/mL3ze6MLMr
-----

```

3.3.4 Running the Apache Server

1. Download custom Apache configuration files (httpd.conf and httpd-ssl.conf)
 - these configure Apache to use only TLS 1.3 with 2 high-security cipher suites with forward secrecy and authenticated encryption with associated data (ECDHE with AES-256 in Galois Counter Mode and SHA-384 hashing, or ECDHE with ChaCha20 and Poly1305). These files also set Apache to announce the NIST Round 3 Level 5 hybrid handshake algorithms to clients

```
-cd Downloads
-wget https://li-article-bucket.s3.us-east-2.amazonaws.com/httpd.conf
-wget https://li-article-bucket.s3.us-east-2.amazonaws.com/httpd-ssl.conf
-sudo mv httpd.conf /usr/local/apache2/conf/httpd.conf
-sudo mv httpd-ssl.conf /usr/local/apache2/conf/extra/httpd-ssl.conf
```

2. Move the generated server certificate and key to appropriate Apache folder from your home working folder

```
deepesh@deepesh:~$ cd Downloads
deepesh@deepesh:~/Downloads$ sudo mv httpd.conf /usr/local/apache2/conf/httpd.conf
deepesh@deepesh:~/Downloads$ sudo mv httpd-ssl.conf /usr/local/apache2/conf/extra/httpd-ssl.conf

deepesh@deepesh:~/Downloads$ cd ~
deepesh@deepesh:~$ sudo mv p521_falcon1024_srv.crt /usr/local/apache2/conf/p521_falcon1024_srv.crt
deepesh@deepesh:~$ sudo mv p521_falcon1024_srv.key /usr/local/apache2/conf/p521_falcon1024_srv.key
```

Start the Apache server running on localhost listening on port 4433(per configuration files)

```
-cd
-sudo mv p521_falcon1024_srv.crt /usr/local/apache2/conf/p521_falcon1024_srv.crt
-sudo mv p521_falcon1024_srv.key /usr/local/apache2/conf/p521_falcon1024_srv.key
```

3. Start Epiphany browser and connect to local Apache server on port 4433

- *Install Epiphany web browser and prerequisites*

```
-sudo apt install build-essential clang meson gnome-pkg-tools libglib2.0-dev libproxy-dev
getsettings-desktop-schemas-dev ca-certificates epiphany-browser -y
```

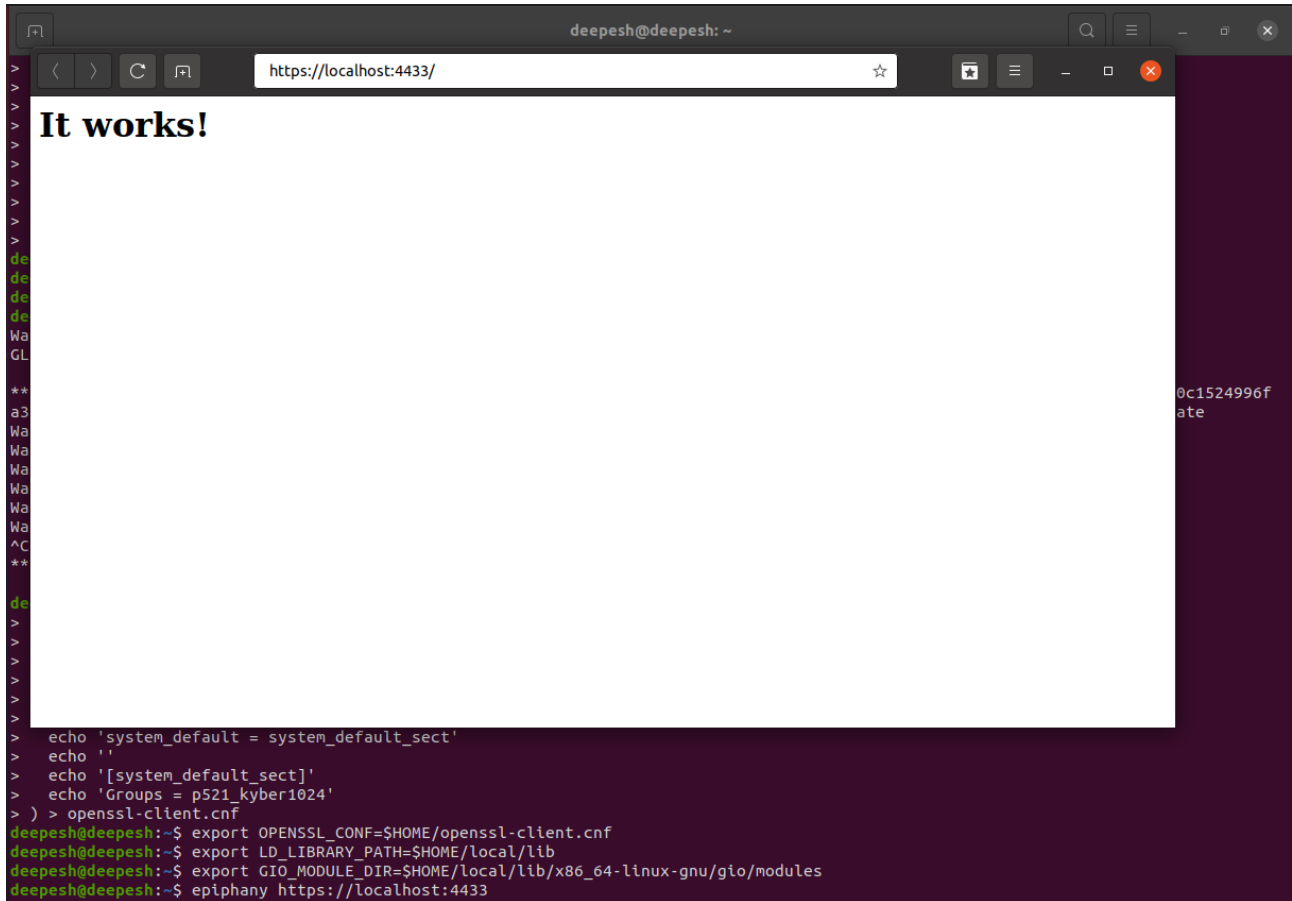
- *Clone GNOME's glib-networking back-end, and configure it to use OQS OpenSSL instead of its built-in GNU TLS library*

```
- cd glib-networking
-mkdir build
-cd build
-env PKG_CONFIG_PATH=$HOME/local/lib/pkgconfig CPATH=$HOME/local/include
LIBRARY_PATH=$HOME/local/lib meson --prefix=$HOME/local -Dopenssl=enabled -
Dgnutls=disabled ..
-env CPATH=$HOME/local/include ninja
-ninja install
-cd
```

- *Create a client OpenSSL configuration file and environment variables which will force the Epiphany browser to use the glib-networking back-end we built, as well as the p521_firesaber hybrid TLS handshake.*

Start Epiphany browser and connect to local Apache server on port 4433

```
-epiphany https://localhost:4433
```



In this experiment, we built an Apache web server and made a quantum safe connection to it with the localhost. We used kyber_1024 for Key-encapsulation scheme, falcon_1024 for signature scheme and P-521 for elliptic curve algorithm.

4 Comparison and Analysis of TLS connections

4.1 Running TLS Demo using Different Algorithms

OpenSSL contains a basic TLS server (s_server) and TLS client (s_client) which can be used to demonstrate and test TLS connections.

So, in order to run a server we have to generate a X.509 certificate first, using either a classical (rsa), quantum-safe or hybrid algorithm(a classical algorithm combined with a quantum safe algorithm).

4.1.1 Generating X.509 self-signed certificate using classical authentication algorithms(here, rsa)

It can be done in two ways :

1. In this part first the server will generate public private key pair and a certificate signing request(CSR) and then send the certificate to the CA.

We can generate key pair by running this command on linux(to run this command you must have openssl configured in your system) :

```
-openssl req -new -newkey rsa:1024 -keyout rsa_1024.key -out rsa_1024.csr -nodes -subj  
"/CN=oqstest server"
```

Here, the req command primarily creates and processes certificate requests.

The *-newkey* option creates a new certificate request and a new private key. The argument takes one of several forms. *rsa:nbits*, where *nbits* is the number of bits, generates an RSA key *nbits* in size. If *nbits* is omitted, i.e. *-newkey rsa* specified, the default key size, specified in the configuration file is used.

More about the options of req command can be found [here](#).

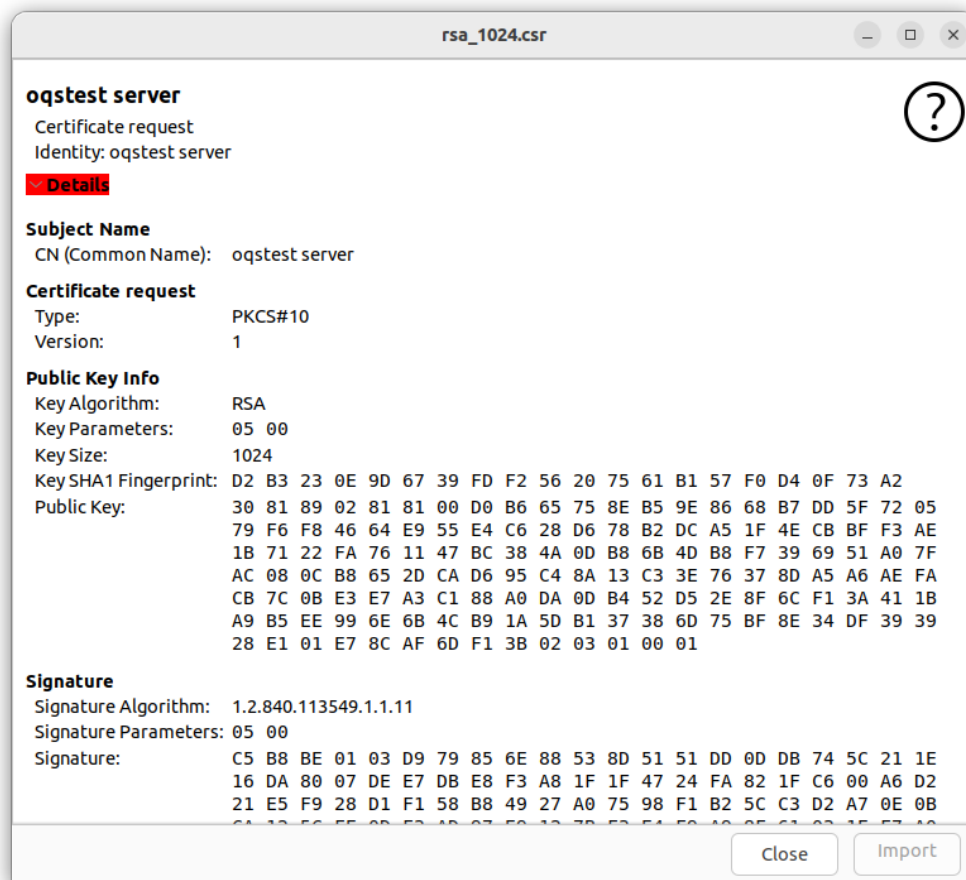
Finally, the private key will be copied to file 'rsa_1024.key' (here we are generating private key of 1024 bits) and the CSR certificate will be copied to 'rsa_1024.csr'

```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl req -new -newkey rsa:1024 -keyout rsa_1024.key  
-out rsa_1024.csr -nodes -subj "/CN=oqstest server"  
.+++++.  
...+++++.-----  
mayank_k@mayank-Laptop:~$
```

The private key will look like this:

```
-----BEGIN PRIVATE KEY-----
MIICdgIBADANBgkqhkiG9w0BAQEFAASCAmAwggJcAgEAAoGBANC2ZXW0tZ6GaLf
dX3IFefb4RmTpVeTGKNZ4stylH07Lv/OuG3Ei+nYRR7w4Sg24a0249zlpUaB/rAg
uGUtytaVxIoTwz52N42lpq76y3wL4+ejwYig2g20UtUuj2zx0kEbqbXumW5rTLka
XbE3OG11v4403zk5K0EB54yvbfe7AgMBAAECgYAmhyUfmGKKiWfVv7XY1I3500ez
ZSeH/dSYEtvhW1G0OCqNDzdm7lkhjgZ1MJG2h5k0JpCr0B2olrdeYuokdV5hfi9
h1iKEC8mkW4J0t+JSgqbngyHKTzGSlxGgZnE8dV3qGHqexJPuX0WhPn16ht1PK0S
dRV73WU/jSRT3wGMIQJBA02K8aoHumEh7zNLz+AAtyWo3DMpmxL7msQoqb5LDqae
rzdil5icLrX9Nyh4rsUka7PJ22a5aBuqeVJVHAKSzaMCQQDg7fptPdW+mMxaTcfE
OrRyBOxcvegZh52+NsNU1lJzL1LLUUHdW4X3WJCeT00MwPHMvr3j3BLSOTCpA8v5
zteJAKAcii8S4K7R8MP0g/NL1Iosqicator9MuGbY89RH6948WPX4a6ZfVq/Xtb7
s9Q7t26BQWVs6CXL/SVqcPNwk9DFakAPA6FIjf2yZG1WH5D7ff7S9k1yCj74SRji
aoS3sz3R0c0+VqLKVzT3ASZha+pxwuSQ4d7I2VSc/vi8DZuVU73pAkeEAtytmRZv6
XbdzAnks10w2ty8MJiSa713q1yQYQwbKscbtQrjyTv60YKy/dau9vQAThSa3WoLU5
JHvbPDLnibajyg==
-----END PRIVATE KEY-----
```

The CSR certificate :



Now in step 2 the server sends the CSR certificate(with public key in it) to the CA and CA will generate the signed server certificate.

The command for this operation is:

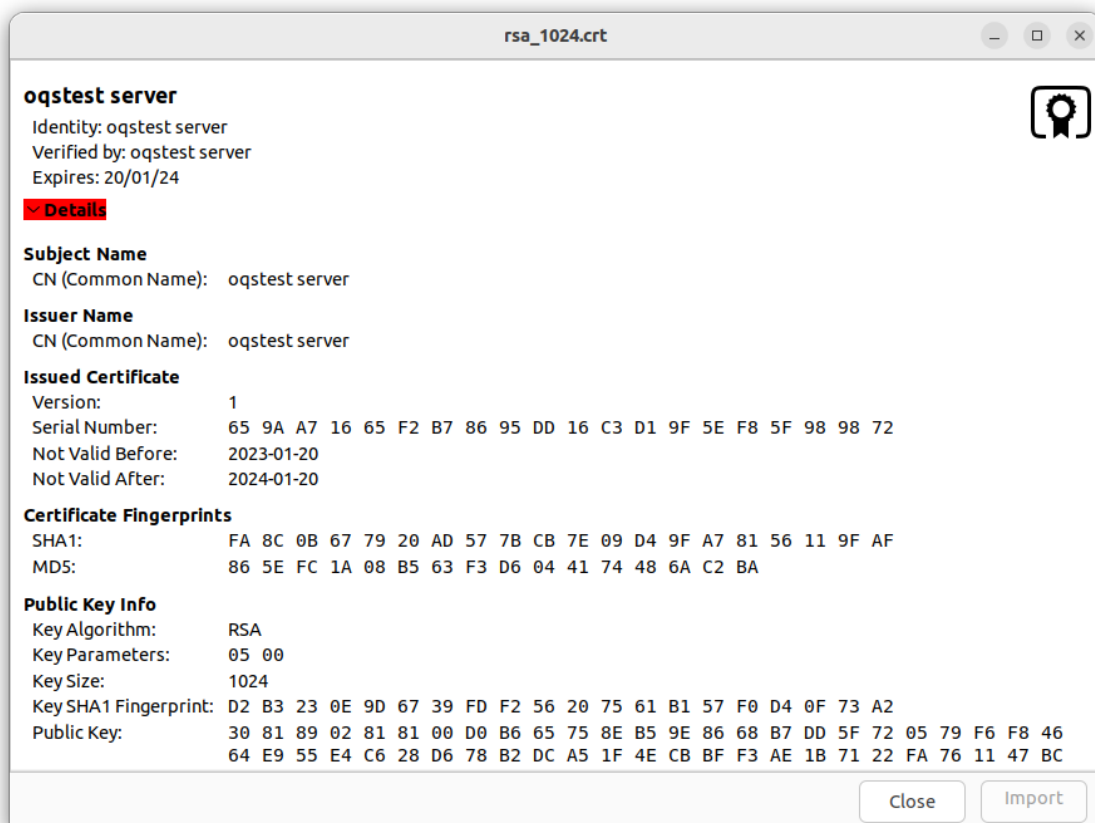
```
-openssl x509 -req -in rsa_1024.csr -signkey rsa_1024.key -out rsa_1024.crt -days 365
```

Here the days indicate the expiry of certificate.

```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl x509 -req -in rsa_1024.csr -signkey rsa_1024.key -out rsa_1024.crt -days 365  
Certificate request self-signature ok  
subject=CN = oqstest server  
mayank_k@mayank-Laptop:~$
```

Here we get the certificate in the file named 'rsa_1024.crt'.

The certificate will look like this:



2. In this part we will directly generate the key pair and certificate in a single command.
`openssl req -x509 -new -newkey rsa -keyout rsa.key -out rsa.crt -nodes -subj "/CN=oqstest CA" -days 365`


```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl s_server -key rsa.key -cert rsa.crt -accept 44330 -www  
Using default temp DH parameters  
ACCEPT  
█
```

Accessing s_server via openssl s_client

-openssl s_client -connect localhost:44330

```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl s_client -connect localhost:44330  
CONNECTED(00000003)  
Can't use SSL_get_servername  
depth=0 CN = oqstest CA  
verify error:num=18:self-signed certificate  
verify return:1  
depth=0 CN = oqstest CA  
verify return:1  
---  
Certificate chain  
 0 s:CN = oqstest CA  
  i:CN = oqstest CA  
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256  
   v:NotBefore: Jan 20 19:30:20 2023 GMT; NotAfter: Jan 20 19:30:20 2024 GMT  
---  
Server certificate  
-----BEGIN CERTIFICATE-----  
MIIDCzCCAF0gAwIBAgIUxsBlUa4zSXvJcOW8ZqUHIIsyqG9IwDQYJKoZIhvcNAQEL  
BQAwFETMBEGA1UEAwKb3FzdGVzdCBBDQTAeFw0yMzAxMjAxOTMwMjBaFw0yNDAx  
MjAxOTMwMjBaMBUXEzARBgNVBAMMCM9xc3Rlc3QgQ0EwggEiMA0GCSqGSIb3DQEB  
AQUAA4IBDwAwggEKAoIBAQCgXydL8KSeX/IkIgPnzc8QLc44AhNjIU7cDmA1a14h  
MjCUiglHPm2Ph9eIv/TslPn00ydIHNJFrndT678Q90NeZhdNWzJIoLHxvIboHpmQ  
0pPTISg+0+QoJH+Igt76JtctLYA1LKGA0Bz6Ca4La8j1Tmua/lm+UCyJP7C8i0By  
8ErB0tsKJ1Vie5LCp9WwhmD1Tnh2befKd8LPbnKEE4fzyvem/81xGXR1tspzFEwp  
09qa7DufIKnS1lOqHiM/toN/XWfB0JLs1UNEz5f5fJNEqqYad3PCIibfT1fjrH6w  
RRJ69KNJHbT/SR9V8N7G9EbSj1txeEOUnrnyF66RIX8nAgMBAAGjUzBRMB0GA1Ud
```

Here, we can see that the local s_client is connected to the server and showing the server certificate and signature algorithm used.

4.1.2 Generating X.509 self-signed certificate using hybrid authentication algorithms

Hybrid authentication algorithms: combining a quantum-safe algorithm listed in here with a traditional digital signature algorithm.

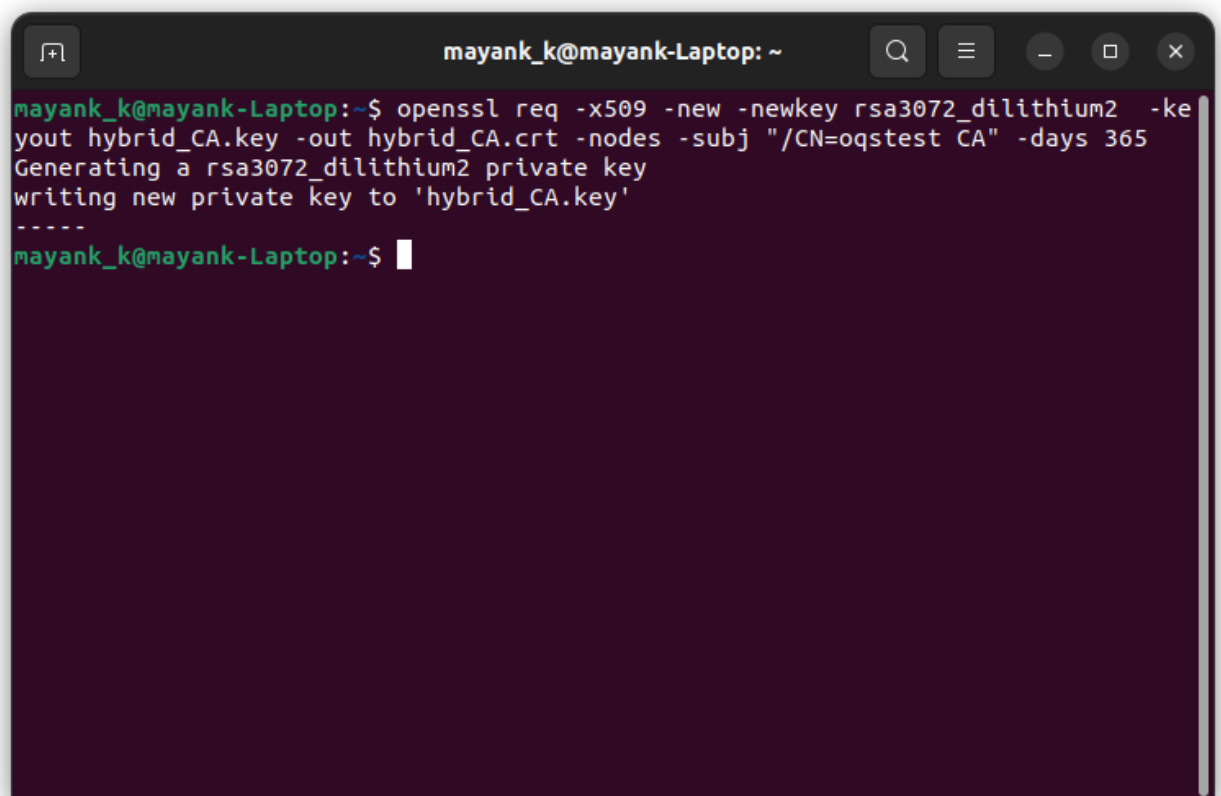
Here, we are using the hybrid(rsa3072_dilithium2) algorithm.

Command to generate a self-signed CA certificate and private key:

```
-openssl req -x509 -new -newkey SIG -keyout SIG_CA.key -out SIG_CA.crt -nodes -subj  
"/CN=oqstest CA" -days 365
```

Here, in place of SIG we can put our own hybrid algorithms. We are using:

```
openssl req -x509 -new -newkey rsa3072_dilithium2 -keyout hybrid_CA.key -out hybrid_CA.crt  
-nodes -subj "/CN=oqstest CA" -days 365
```



```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl req -x509 -new -newkey rsa3072_dilithium2 -keyout hybrid_CA.key -out hybrid_CA.crt -nodes -subj "/CN=oqstest CA" -days 365  
Generating a rsa3072_dilithium2 private key  
writing new private key to 'hybrid_CA.key'  
-----  
mayank_k@mayank-Laptop:~$
```

After running this command we can find the private key in file 'hybrid_CA.key' and certificate in 'hybrid_CA.crt'.

Now, The root CA certificate can be used directly to start the server or can be used to issue a server certificate.

Running a basic TLS server

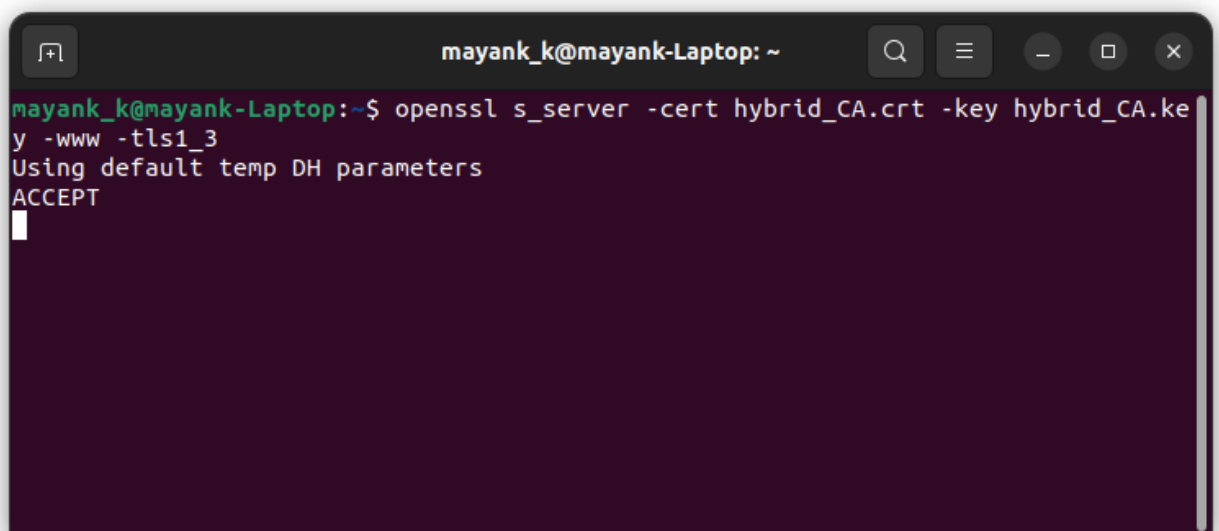
Here we are running the TLS server with all possible key-exchange algorithms enabled using the key and certificate we have generated in previous part.

Command to run s_server:

```
-openssl s_server -cert SERVER.crt -key SERVER.key -www -tls1_3
```

In the given command in place of SERVER we can put our file names of key and certificate. In our case it is:

```
openssl s_server -cert hybrid_CA.crt -key hybrid_CA.key -www -tls1_3
```

A terminal window titled 'mayank_k@mayank-Laptop: ~' with standard window controls. The command 'openssl s_server -cert hybrid_CA.crt -key hybrid_CA.key -www -tls1_3' is entered and executed. The output shows 'Using default temp DH parameters' followed by 'ACCEPT' on a new line, with a cursor at the end of the line.

```
mayank_k@mayank-Laptop:~$ openssl s_server -cert hybrid_CA.crt -key hybrid_CA.key -www -tls1_3
Using default temp DH parameters
ACCEPT
```

Accessing s_server via openssl s_client

In another terminal window, we can run a TLS client requesting one of the supported key-exchanges listed here.

```
-openssl s_client -groups KEX -CAfile SIG_CA.crt
```

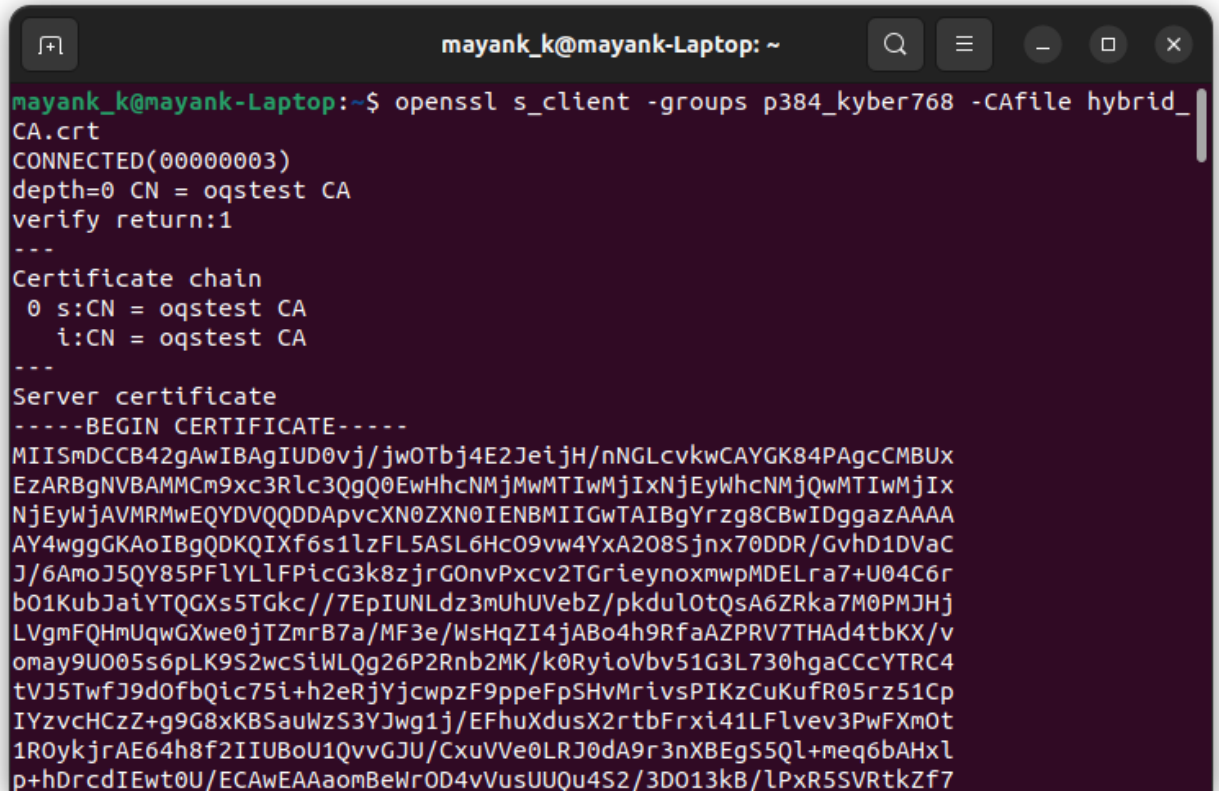
KEX = one of the quantum-safe or hybrid key exchange algorithms

So, here we are using p384.kyber768 key exchange algorithm. We will be running command in another terminal than the terminal in which s_server is running:

-openssl s_client -groups p384_kyber768 -CAfile hybrid_CA.crt

-CAfile file: A file containing trusted certificates to use during server authentication and to use when attempting to build the client certificate chain.

Now, we can see the certificate of server and the algorithms used in s_client.

A terminal window titled 'mayank_k@mayank-Laptop: ~' with standard window controls. The terminal shows the command 'openssl s_client -groups p384_kyber768 -CAfile hybrid_CA.crt' being executed. The output includes connection status, depth, CN, verify return, certificate chain details, and a full server certificate in PEM format.

```
mayank_k@mayank-Laptop:~$ openssl s_client -groups p384_kyber768 -CAfile hybrid_CA.crt
CONNECTED(00000003)
depth=0 CN = oqstest CA
verify return:1
---
Certificate chain
 0 s:CN = oqstest CA
  i:CN = oqstest CA
  ---
Server certificate
-----BEGIN CERTIFICATE-----
MIISmDCCB42gAwIBAgIU0vj/jwOTbj4E2Jei jH/nNGLcvkwCAYGK84PAgcCMBUx
EzARBgNVBAMMcm9xc3Rlc3QgQ0EwHhcNMjMwMTIwMjIxNjEyWhcNMjQwMTIwMjIx
NjEyWjAVMRMwEQYDVQDDApvcXN0ZXN0IENBMiIGwTAIBgYr zg8CBwIDggazAAAA
AY4wggGKAoIBgQDKQIXf6s1lzFL5ASL6Hc09vw4YxA208Sjnx70DDR/GvhD1DVaC
J/6AmoJ5QY85PFLYLFPicG3k8zjrG0nvPxcv2TGrieynoxmwpMDELra7+U04C6r
b01KubJaiYTQGXs5TGkc//7EpIUNLdz3mUhUVebZ/pkduLotQsA6ZRka7M0PMJHj
LVgmFQHmUqwGXwe0jTZmrB7a/MF3e/WSHqZI4jABo4h9RfaAZPRV7THAd4tbKX/v
omay9U005s6pLK9S2wcSiWLQg26P2Rnb2MK/k0RyioVbv51G3L730hgaCCcYTRC4
tVJ5TwfJ9d0fbQic75i+h2erjYjcw pzf9ppeFpSHvMrivsPIKzCuKufR05rz51Cp
IYzvcHCZz+g9G8xKBSauWzS3YJwg1j/EFhuXdusX2rtbFrxi41LFlvev3PwFXmOt
1R0ykjrAE64h8f2IIUBoU1QvvGJU/CxuVVe0LRJ0dA9r3nXBEgS5Ql+meq6bAHxL
p+hDr cdiEwt0U/ECAwEAAaomBeW rOD4vVusUUQu4S2/3D013kB/LPxR5SVRtkZf7
```

```
mayank_k@mayank-Laptop: ~  
-----BEGIN CERTIFICATE-----  
MIIDBTCCAeGgAwIBAgIQDzZKZ2p4g5S0T0K41jyCL0NtEgcylCkGow  
hVp6cg7rD0NE4+Qcfs9DggioPKaIR7cAQY0JEFJdHeUoKm6wNjj5+4NRUxYWL1i  
bKC0xMnO6u4JISVDRktVXl9jhqus2+fyAys6g4ulrtXd6AAAAAAAAAAAAAAAAAA  
AAAAAAAAAARIDA6  
-----END CERTIFICATE-----  
subject=CN = oqstest CA  
  
issuer=CN = oqstest CA  
  
---  
No client certificate CA names sent  
Peer signature type: RSA3072 - Dilithium2  
Server Temp Key: p384_kyber768 hybrid  
---  
SSL handshake has read 9053 bytes and written 1688 bytes  
Verification: OK  
---  
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384  
Server public key is 13712 bit  
Secure Renegotiation IS NOT supported  
Compression: NONE  
Expansion: NONE  
No ALPN negotiated  
Early data was not sent  
Verify return code: 0 (ok)
```

4.2 Comparing time for TLS handshake with different key exchange algorithms (Performance Testing)

In previous section we have made X.509 certificates for different algorithms and run a TLS basic server. In this section we will be using those certificates to make TLS connections and calculate number of connections per second and thus how TLS handshake time.

I. USING CLASSICAL ALGORITHMS (RSA)

We will be using RSA algorithm certificate to run s_server and make our TLS connection, To run the server we will run command (using rsa.key and rsa.cert files):

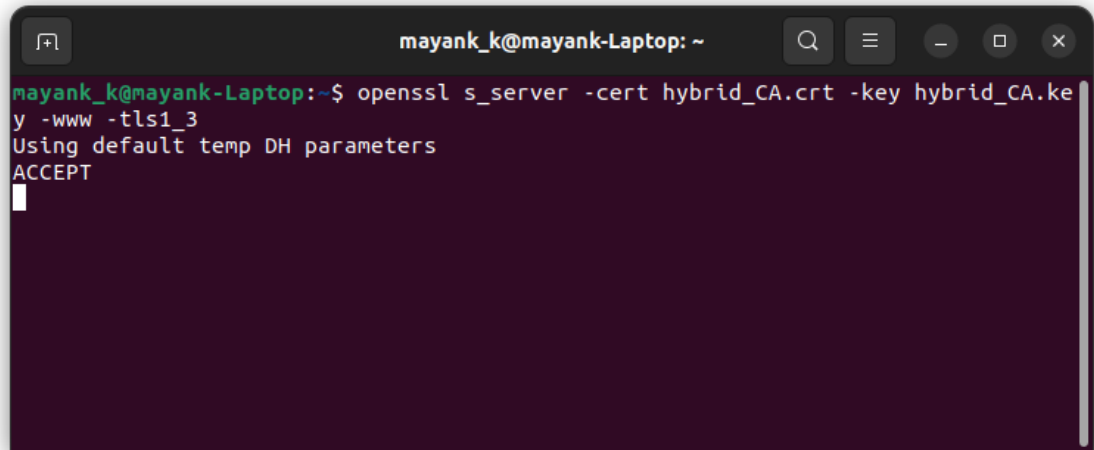
```
-openssl s_server -key rsa.key -cert rsa.crt -accept 44330 -www
```

II. USING HYBRID ALGORITHMS

Using certificate of rsa3072_dilithium2 .

Run s_server command:

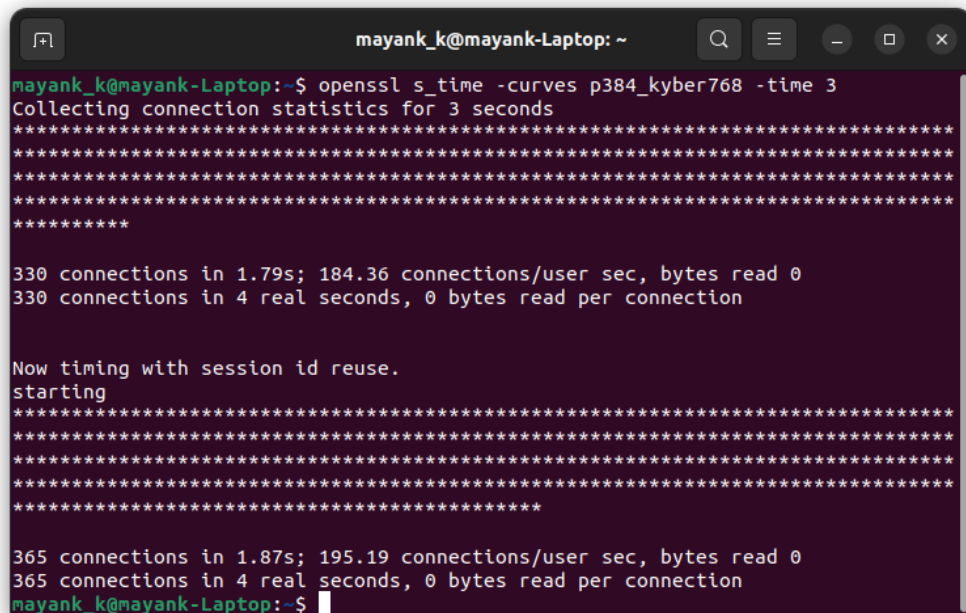
`-openssl s_server -cert hybrid_CA.crt -key hybrid_CA.key -www -tls1_3`



```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl s_server -cert hybrid_CA.crt -key hybrid_CA.key -www -tls1_3  
Using default temp DH parameters  
ACCEPT  
█
```

Accessing server via command:

`-openssl s_time -curves p384_kyber768 -time 3`



```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl s_time -curves p384_kyber768 -time 3  
Collecting connection statistics for 3 seconds  
*****  
*****  
*****  
*****  
*****  
  
330 connections in 1.79s; 184.36 connections/user sec, bytes read 0  
330 connections in 4 real seconds, 0 bytes read per connection  
  
Now timing with session id reuse.  
starting  
*****  
*****  
*****  
*****  
*****  
  
365 connections in 1.87s; 195.19 connections/user sec, bytes read 0  
365 connections in 4 real seconds, 0 bytes read per connection  
mayank_k@mayank-Laptop:~$ █
```

Here we can see that, the number of connections per sec is 195 which is less than the connections when we are using classical algorithms.

III. USING ONLY QUANTUM SAFE ALGORITHMS

Here we are using dilithium2 certificate to run the server.

```
-openssl s_server -cert dilithium2.crt -key dilithium2.key -www -tls1_3
```

```
mayank_k@mayank-Laptop: ~  
mayank_k@mayank-Laptop:~$ openssl s_server -cert dilithium2.crt -key dilithium2.  
key -www -tls1_3  
Using default temp DH parameters  
ACCEPT
```

Accessing server via command:

```
-openssl s_time -curves kyber768 -time 3
```

```
mayank_k@mayank-Laptop: ~  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
  
3291 connections in 1.84s; 1788.59 connections/user sec, bytes read 0  
3291 connections in 4 real seconds, 0 bytes read per connection  
mayank_k@mayank-Laptop:~$
```

Here we can see that the average connections per sec are 1173 which are a bit more than the case of hybrid algorithms but less than the case of classical algorithms.

So on concluding note we can say about TLS handshake times in general at same security level and same group of algorithms in the order:

Hybrid > Pure Quantum Safe > Classical

4.2.1 Comparison Table of number of TLS handshakes using different combinations of Signature and Key exchange algorithms

Running servers with different X.509 certificates signed using different signature algorithms and then accessing the server with different key exchange algorithms.

	Digital Signature Algorithm (SIG)	Key Exchange Algorithms (KEM)	Claimed NIST level		Public key size(bytes)		TLS connections/s
			SIG	KEM	SIG	KEM	
Open quantum safe algorithms (OQS)	dilithium2	kyber768	2	3	1312	1184	1685
	falcon1024	kyber1024	5	5	1793	1568	1094
	dilithium2	bikel3	2	3	1312	3083	147
	dilithium3	bikel3	3	3	1952	3083	143
	dilithium5_aes	frodo640aes	5	1	2592	9616	386
Hybrid Algorithms (Integrating classical with oqs algorithms)	rsa3072_dilithium2	p384_kyber768	2	3	1696	1232	256
	rsa3072_falcon512	p256_hqc128	1	1	1281	2281	807
	p521_falcon1024	p256_hqc128	5	1	1859	2281	282
	p384_dilithium3_aes	p521_kyber90s1024	3	5	2000	1634	99
	p256_sphincsharaka128frobust	p521_kyber90s1024	1	5	64	1634	116

From the table we can say that the with increase in public key size in same group of algorithms, the speed of connections will decrease. It will take more time to exchange the keys.

5 Conclusion and Future Scope

The current encryption algorithms, including public-key cryptography, are believed to be secure for conducting e-commerce transactions. Although quantum computing is a real threat, it is still a relatively expensive technology, and its applications are mainly limited to scientific and government research. Nonetheless, researchers are competing to develop post-quantum encryption protocols that can withstand quantum attacks, while other researchers are working on quantum algorithms to break RSA and other existing cryptosystems. This creates a race between those trying to develop stronger encryption methods and those trying to break the existing ones with quantum computing. In conclusion, post-quantum cryptography is an important area of research that will become increasingly important as quantum computers become more powerful. While many challenges remain, researchers are making progress in developing post-quantum encryption schemes that are both secure and efficient. In the future, we can expect to see the widespread adoption of post-quantum cryptography to secure sensitive data in a quantum-safe manner.

The future scope for post-quantum cryptography (PQC) is vast and promising. Here are a few potential areas where PQC could play a significant role:

1. **Secure communication:**PQC will play a crucial role in ensuring secure communication in the future. As quantum computers become more powerful, they will be able to break many of the encryption schemes that are currently used to secure sensitive data. PQC will provide a quantum-safe alternative to these encryption schemes and will ensure that sensitive information remains secure.
2. **E-commerce:**With the rise of e-commerce, the need for secure online transactions has never been greater. PQC will provide a more secure way to conduct online transactions, ensuring that sensitive financial data remains protected from quantum-based attacks.
3. **Government agencies:**Government agencies deal with sensitive information on a daily basis, and it is crucial that this information remains secure. PQC will provide a quantum-safe way to protect this information and ensure that it cannot be accessed by unauthorized parties.
4. **Healthcare:**The healthcare industry deals with sensitive patient data on a regular basis, and it is crucial that this data remains secure. PQC will provide a way to secure this data and ensure that it cannot be accessed by unauthorized parties, protecting patients' privacy and confidentiality.

Overall, PQC has the potential to revolutionize the way we secure sensitive data and communications, providing a quantum-safe alternative to existing encryption schemes. As quantum computers become more powerful, the need for PQC will only increase, making it a vital area of research and development in the coming years.

References

- Douglas Stebila, Michele Mosca. Post-quantum key exchange for the Internet and the Open Quantum Safe project. In Roberto Avanzi, Howard Heys, editors, *Selected Areas in Cryptography (SAC) 2016*, LNCS, vol. 10532, pp. 1–24. Springer, October 2017. <https://openquantumsafe.org> accessed 2023
- Celi, Sofía, and Thom Wiggers. "KEMTLS: post-quantum TLS without signatures, January 2020."
- Schwabe, Peter, Douglas Stebila, and Thom Wiggers. "Post-quantum TLS without handshake signatures." *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2020.